



UNIVERSITY OF PADOVA

Department of Mathematics “Tullio Levi-Civita”

Master’s Degree in Data Science

Projected Gradient and Frank-Wolfe Variants for Portfolio Optimization

Professors

Francesco Rinaldi
Gennaro Auricchio

Students

Giorgia Bonin, 2142585
Emanuele Cavaliero, 2164274
Francesco Ceron, 2158534

Academic Year 2024/2025

Contents

1	Introduction	2
2	Modern Portfolio Theory	2
3	Methods	3
3.1	Projected Gradient Descent	3
3.1.1	Optimality gap	3
3.1.2	Projection Algorithm	4
3.2	Frank-Wolfe Algorithm	4
3.2.1	Duality gap	5
3.2.2	Remarks	5
3.3	Away-step FW (AFW)	6
3.4	Pairwise Frank-Wolfe (PFW)	6
3.5	Stepsize	7
3.6	Convergence of the classical FW	7
3.6.1	Curvature and convergence.	8
3.6.2	Linear convergence under stronger assumptions	8
3.6.3	Sparsity and iterates	8
4	Datasets	9
4.1	Market description	9
4.2	Properties of the problem	9
4.3	The role of the condition number in optimization problems	10
5	Experimental Results	10
5.1	Convergence analysis	10
5.1.1	Frank-Wolfe and variants	10
5.1.2	Projected Gradient Descent	14
5.1.3	CPU-time comparisons	15
5.2	Sparsity	16
6	Conclusion	16
7	Appendix	18
7.1	Derivation of the exact line search.	18
7.2	Convergence Plots	18
7.3	Convergence of all methods	32
7.4	Stepsizes plots	34
7.5	CPU-time comparisons	36
7.6	Portfolio Allocations	39

1 Introduction

The following report aims to contribute to the constrained optimization framework, by analyzing projection-based and projection-free first-order methods for mean-variance Portfolio Optimization over the probability simplex. We theoretically implement and compare four algorithms: Projected Gradient Descent (PGD), Frank-Wolfe (FW), Pairwise Frank-Wolfe (PFW) and Away-step Frank Wolfe (AFW), under the common quadratic, convex objective with non-negativity and budget constraints. The empirical framework uses five real equity universes (FTSE100, EUROSTOXX50, DOWJONES, S&P500 and NASDAQ100) with different sizes and posing different challenges for the optimization task. We first describe the problem from a theoretical perspective, with the aim of setting the basis on which the optimization problem is solved. We then benchmark convergence behavior (objective decay, optimality certificates), computational cost (CPU time), and final portfolio structure (sparsity of the final allocations). We conclude with practical recommendations on method/stepsize selection for problems that look similar to the one we solved, further discussing limitations and natural extensions.

2 Modern Portfolio Theory

Modern Portfolio Theory (MPT), also known as mean-variance analysis, is a mathematical framework introduced by economist Harry Markowitz in 1952 as a means of assembling a portfolio of assets in an optimal way. The objective is to maximize the expected return for a given level of risk, or equivalently, to minimize risk at a desired level of expected return (Markowitz, 1952).

Given a total budget b and n assets, the Markowitz problem can be formulated as follows:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & \eta x^\top \Sigma x - \bar{r}^\top x \\ \text{s.t.} \quad & \sum_{i=1}^n x_i = b, \\ & x \geq 0. \end{aligned} \tag{1}$$

where:

- x : vector of portfolio asset weights, i.e., the amount of money invested in each asset. Two constraints apply:
 1. the sum of the weights must be equal to the total budget b
 2. the weights must be greater than or equal to 0, ruling out short selling
- Σ : covariance matrix of asset returns.
- η : investor's level of risk aversion. It ranges from 0 to $+\infty$, with $\eta = 1$ optimizing the balance between expected return and risk, $0 < \eta < 1$ representing risk-tolerant investors and $\eta > 1$ indicating risk-averse investors.
- $\bar{r}$: vector of expected returns for each asset.

This results in a convex quadratic constrained optimization problem, which can be written in the form:

$$\min_{x \in C} f(x) \quad C = \text{conv}(\mathcal{A}) \tag{2}$$

where, by setting $b = 1$, the feasible set C becomes the probability simplex, i.e., the $(n - 1)$ -dimensional simplex whose vertices are the n standard unit vectors in $\mathbb{R}^n$.

The gradient of the problem is given as follows:

$$\nabla f(x) = 2\eta \Sigma x - \bar{r}. \quad (3)$$

3 Methods

In this section, the Projected Gradient method, the Conditional Gradient method (also known as the Frank-Wolfe) and its variants are described.

3.1 Projected Gradient Descent

The Projected Gradient is a variant of the classical Gradient Descent method commonly employed in constrained optimization frameworks. While for unconstrained optimization problems it is sufficient to find a descent direction, in constrained optimization the next iterate is required to be feasible. The traditional Gradient method might suggest moving to a point $x_{k+1} \notin C$, violating constraints. The issue is overcome by choosing the point in C closest to $x_{k+1} := x_k - s_k \nabla f(x_k)$, with $s_k > 0$. This new point is selected by taking the projection of x_{k+1} onto C , i.e., $\hat{x}_k := \Pi_C(x_k - s_k \nabla f(x_k))$ [11].

The algorithm of the method is thus given as follows:

Algorithm 1 Projected Gradient Descent (PGD) Scheme [11]

- 1: Choose an initial point $x_1 \in C$, a tolerance $\epsilon > 0$ and a maximum number of iterations K
 - 2: **for** $k = 1, 2, \dots$ **do**
 - 3: Set $\hat{x}_k = \Pi_C(x_k - s_k \nabla f(x_k))$, with $s_k > 0$
 - 4: **if** $\|g_C(x_k)\| \leq \epsilon$ **then**
 - 5: **stop**
 - 6: **end if**
 - 7: Update $x_{k+1} = x_k + \gamma_k(\hat{x}_k - x_k)$, with $\gamma_k \in (0, 1]$ suitably chosen
 - 8: **end for**
-

3.1.1 Optimality gap

The optimality condition is implemented as follows:

$$\|g_C(x_k)\| \leq \epsilon, \quad \text{where } g_C(x_k) := x_k - \Pi_C(x_k - \nabla f(x_k)),$$

and $\epsilon > 0$ is a tolerance. Note that $\|g_C(x_k)\| = 0$ if and only if x_k is a stationary point, since, as described in [11], x^* is optimal for 2 if

$$x^* = \Pi_C(x^* - s_k \nabla f(x^*)), \quad s_k > 0,$$

meaning that, performing a gradient step and projecting the point back onto the feasible set results in the same point.

3.1.2 Projection Algorithm

In our analysis, we considered both the algorithm proposed by Duchi in [5] for the Euclidean projection onto the simplex and the algorithm proposed by Condat [4]. We adopt Condat’s algorithm, as it provides significantly faster runtime, improving computational efficiency while solving the same problem.

Algorithm	Time (s)	Sum of projected vector
Duchi	12.6427	1.0
Condat	3.2164	1.0

Table 1: Execution times for simplex projection with $n = 10^8$

3.2 Frank-Wolfe Algorithm

The Frank-Wolfe (FW) algorithm, also known as the conditional gradient method, was originally introduced by Marguerite Frank and Philip Wolfe [6] for quadratic optimization over polytopes. It is designed for constrained optimization problems and has recently regained popularity due to its projection-free nature. At every step, the algorithm moves towards the so-called FW vertex, by minimizing a linearized objective, generating feasible iterates [2]. When C is the simplex, the linear minimization oracle (LMO) reduces to selecting the vertex with the smallest gradient component.

Formally, for the standard simplex

$$\Delta_n = \{x \in \mathbb{R}^n : x \geq 0, \mathbf{1}^\top x = 1\} = \text{conv}\{e_i \mid i = 1, \dots, n\}.$$

the LMO has the closed form

$$s_k \in \arg \min_{s \in \Delta_n} \langle g_k, s \rangle = e_{i_k}, \quad i_k \in \arg \min_{i \in \{1, \dots, n\}} [g_k]_i.$$

where e_{i_k} denotes the i_k -th canonical basis vector [11].

The general FW scheme can be summarized as follows:

Algorithm 2 Frank–Wolfe (FW) Scheme

- 1: Choose an initial point $x_1 \in C$, a tolerance $\epsilon > 0$, and a maximum number of iterations K .
- 2: **for** $k = 1, 2, \dots, K$ **do**
- 3: Compute the gradient $g_k = \nabla f(x_k)$.
- 4: Solve the Linear Minimization Oracle (LMO):

$$s_k = \arg \min_{s \in C} \langle g_k, s \rangle.$$

- 5: Compute the FW direction $d_k = s_k - x_k$.
 - 6: Compute the duality gap $G(x_k) = \langle g_k, x_k - s_k \rangle$ (equivalently, $G(x_k) = \langle -g_k, d_k \rangle$).
 - 7: **if** $G(x_k) \leq \epsilon$ **then**
 - 8: **stop**
 - 9: **end if**
 - 10: Choose a suitable stepsize $\gamma_k \in (0, 1]$.
 - 11: Update $x_{k+1} = x_k + \gamma_k d_k$.
 - 12: **end for**
 - 13: **return** x_{k+1}
-

3.2.1 Duality gap

Since the primal error $f(x) - f(x^*)$ is intractable, the FW algorithm uses the duality gap as an optimality measure.

For a feasible point $x \in C$, the duality gap is defined as

$$G(x) = \max_{s \in C} \langle g_k, x - s \rangle \quad (4)$$

As shown in [2], this quantity is always non-negative and upper bounds the primal error. Therefore, if $G(x)$ is sufficiently small, x is guaranteed to be close to the optimal solution. Importantly, this certificate comes at no additional computational cost, since it is given as a byproduct of the LMO at each FW iteration.

Moreover, as reported in [2], on a convex and closed subset $B \subseteq \mathbb{R}^n$, the Frank–Wolfe gap is related to the projected gradient optimality condition as follows:

$$\|g_C(x_k)\|^2 \leq G(x_k). \quad (5)$$

3.2.2 Remarks

The FW algorithm is simple to implement and well-suited to large-scale problems involving structured domains. It provides sparse solutions by adding at most one new vertex to the convex combination representing the current solution at each step [2]. However, the standard FW method is known to have a relatively slow sublinear convergence rate of $O(1/k)$ for general convex functions, particularly when the optimum lies on the boundary of the feasible set, which may lead to zig-zagging behavior [9].

To mitigate this issue, several variants have been proposed in the literature, which achieve faster (linear) convergence under stronger assumptions [9].

3.3 Away-step FW (AFW)

Away-step FW is an extension of the standard FW method that also allows moving away from a bad vertex already contained in the active set S_t . At each iteration, the method computes:

- the usual FW vertex $s_k = \arg \min_{s \in C} \langle g_k, s \rangle$,
- the away vertex $v_k = \arg \max_{v \in S_k} \langle g_k, v \rangle$

The algorithm then decides whether to use the FW direction $d^{FW} = s_k - x_k$ or the away direction $d^A = x_k - v_k$, choosing the one that yields the greatest improvement. This mechanism enables AFW to reduce the weight of bad vertices and shrink the active set, thereby achieving faster convergence by limiting the zig-zagging behavior when implemented with a line search.

The scheme of this variant is:

Algorithm 3 Away-step Frank-Wolfe (AFW) Scheme [9]

- 1: Let $x_1 \in C$, set $S_1 := \{x_1\}$, (so that $a_v^{(0)} = 1$ for $v = x^{(0)}$ and 0 otherwise), fix tolerance ϵ and max iterations K
 - 2: **for** $k = 1, 2, \dots, K$ **do**
 - 3: Compute:
 - the gradient $g_k = \nabla f(x_k)$
 - FW vertex $s_k = \arg \min_{s \in C} \langle g_k, s \rangle$
 - Away vertex $v_k \in \arg \max_{v \in S_k} \langle g_k, v \rangle$
 - 4: Define $d^{FW} = s_k - x_k$ and $d^A = x_k - v_k$
 - 5: Compute FW gap $G^{FW}(x_k) = \langle -g_k, d^{FW} \rangle$; **if** $G^{FW}(x_k) \leq \epsilon$ **then stop**
 - 6: **if** $\langle -g_k, d^{FW} \rangle \geq \langle -g_k, d^A \rangle$ **then**
 - 7: $d_k := d^{FW}$, $\gamma_{\max} := 1$
 - 8: **else**
 - 9: $d_k := d^A$, $\gamma_{\max} := \frac{\alpha_{v_k}}{1 - \alpha_{v_k}}$
 - 10: **end if**
 - 11: Choose a suitable stepsize: $\gamma = \min(\gamma_k, \gamma_{\max})$
 - 12: Update $x_{k+1} = x_k + \gamma d_k$
 - 13: Update active set $S_{k+1} := \{v \in A \mid \alpha_v^{(k+1)} > 0\}$
 - 14: **end for**
-

3.4 Pairwise Frank-Wolfe (PFW)

Pairwise Frank-Wolfe is another variant of the standard FW method, the key idea of which is to reallocate weight between the best (i.e., FW) vertex and the worst (i.e., Away) vertex at each iteration. At iteration k , the method defines the search direction as: $d_k^{PFW} := d_k^{FW} + d_k^A = s_k - v_k$ with the maximum feasible stepsize $\gamma_{\max} = \alpha_{v_k}$, where α_{v_k} is the weight of the away vertex v_k in the current convex decomposition as described in [9]. From an algorithmic point of view, PFW follows the same structure as AFW, with lines 6-10 of Algorithm 3.3 replaced by $d_k^{PFW} := s_k - v_k$ and the update rule given by:

$$x_{k+1} = x_k + \gamma d_k^{PFW}, \quad \gamma \in [0, \gamma_{\max}].$$

Although the theoretical convergence rate of PFW is in general less tight than of AFW, empirical evidence shows that it often performs better in practice (see [9] for further details).

Moreover, the pairwise direction plays a central role in the convergence analysis of other FW variants, as highlighted in Equation (6) of [9].

As for classical FW, AFW and PFW convergence is reached when the duality gap falls below a predefined threshold.

3.5 Stepsize

Two different stepsize strategies are implemented for PGD:

- fixed, Lipschitz-dependent stepsize : $\gamma_k = \min(\frac{1}{L}, \gamma_{\max})$
- exact line search : $\gamma_k = \min_{\gamma \in [0, \gamma_k^{\max}]} \arg \min \varphi(\gamma)$ with $\varphi(\gamma) := f(x_k + \gamma d_k)$,

Similarly, two strategies are implemented for each FW variant:

- diminishing stepsize: $\gamma_k = \min(\frac{2}{k+1}, \gamma_{\max})$
- exact line search (see PGD stepsize)

Exact line search for our problem admits a closed-form solution with the following form:

$$\gamma^* = -\frac{\nabla f(x)^\top d}{2\eta d^\top \Sigma d}$$

For detailed calculations, we refer the reader to the Appendix 7.1.

For PGD and the original FW $\gamma_{\max}$ is always fixed at 1, while it changes for AFW and PFW, in the following way:

- AFW : $\gamma_{\max} = 1$ when $d = d^{FW}$, and $\gamma_{\max} = \frac{\alpha_v}{1-\alpha_v}$ when $d = d^A$, where a_{v_t} is the weight of the away atom v . Note that when $\gamma = \gamma_{\max}$ the algorithm performs a 'drop step', reducing the active set size $|S_{t+1}| < |S_t|$.
- PFW: $\gamma_{\max} = a_{v_t}$. Here, when $\gamma = \gamma_{\max}$ we can either have a drop step if $|S_{t+1}| < |S_t|$, or a 'swap step' (i.e., the mass is fully swapped from the away atom to the FW atom) if $|S_{t+1}| = |S_t|$.

3.6 Convergence of the classical FW

For smooth convex objectives, the standard FW algorithm achieves a sublinear convergence rate. Let f be a convex function and L -smooth over a compact convex domain C , and denote by x^* the optimal point. Then, after k iterations, with $k \geq 1$ the FW iterate satisfies:

$$f(x_k) - f(x^*) \leq \frac{2LD^2}{k+1} \quad (6)$$

where $D = \max_{x,y \in C} \|x - y\|$ is the diameter of the domain. This result proves the $O(1/k)$ convergence rate for the Frank-Wolfe method [8, 2].

3.6.1 Curvature and convergence.

The convergence analysis of FW relies on the so-called Curvature constant C_f , defined as:

$$C_f := \sup_{\substack{x, s \in \mathcal{D} \\ \gamma \in [0, 1]}} \frac{2}{\gamma^2} \left(f(y) - f(x) - \langle y - x, \nabla f(x) \rangle \right), \quad y = x + \gamma(s - x). \quad (7)$$

For L -smooth convex functions, it holds that the deviation from the linear approximation (i.e., the Bregman divergence in (7)) grows at most quadratically. As highlighted in [8], one can show that $C_f \leq LD^2$, where D is the diameter of the feasible domain C .

Therefore, the classical FW convergence rate can be expressed as

$$f(x_k) - f(x^*) \leq \frac{2C_f}{k+1} \leq \frac{2 \text{diam}_{\|\cdot\|}(\mathcal{D})^2 L}{k+1}, \quad \forall k \geq 1, \quad (8)$$

This result establishes the sublinear $O(1/k)$ rate under the standard assumptions of convexity, smoothness, and compactness of the domain.

3.6.2 Linear convergence under stronger assumptions

While the classical FW method has a worst-case sublinear convergence rate (see (8)), it can converge linearly for strongly convex functions where the optimum lies on the relative interior of the polytope (i.e., $x^* \in \text{ri}(C)$) and the stepsize is given by exact line search [2]. On the other hand, with the same stepsize strategy and stronger assumptions, FW variants enjoy a linear rate. In particular, AFW and PFW achieve geometric linear convergence when f is μ -strongly convex and the feasible set is represented by a polytope (such as the unit simplex in portfolio optimization). Formally, there exists $\rho \in (0, 1)$ such that:

$$f(x_k) - f(x^*) \leq (1 - \rho)^k (f(x_0) - f(x^*)) \quad (9)$$

where the constant ρ depends on problem-specific parameters (e.g., the Lipschitz constant, the strong convexity constant, and the eccentricity of the feasible set C) [9].

3.6.3 Sparsity and iterates

Since C is the probability simplex, sparsity is directly linked to the number of active assets in the final portfolio x^* (i.e., the support size $|\text{supp}(x^*)|$). Every FW step can add at most one new vertex to the convex combination of atoms, resulting in sparse iterates as cited in [9]. [8] proved that in the quadratic model $f(x) = \|x\|_2^2$ over Δ_n , the following holds for any $1 \leq k \leq n$:

$$\min_{x \in \Delta_n, \text{card}(x) \leq k} \|x\|_2^2 = \frac{1}{k}.$$

and any k -sparse x has a duality gap lower bounded by:

$$g(x) \geq \frac{2}{k}.$$

These results establish a worst-case trade-off between sparsity (i.e., the number of nonzero components) and approximation quality (related to the final value of the function): in other words, the complexity in terms of the number of vertices used by Frank-Wolfe is worst-case optimal on the simplex. Variants, as explained in [8, 9, 2] may accelerate the pruning of the active set in practice, but cannot improve on these limits imposed by the worst case.

4 Datasets

In our empirical framework, we employ five real-world equity datasets, namely FTSE100, Eurostoxx_50, DOWJONES, S&P500, and NASDAQ100 [3]. All datasets consist of daily adjusted prices collected over a common time span from October 6, 2006 to February 24, 2023.

4.1 Market description

- **FTSE 100 (UK)**: the leading index of the London Stock Exchange, representing the 100 largest companies by market capitalization listed in the UK.
- **Euro Stoxx 50 (EU)**: a pan-European index that tracks 50 blue-chip companies from 11 Eurozone countries.
- **Dow Jones Industrial Average (US)**: one of the oldest and most recognized stock indices, composed of 30 major US companies.
- **S&P 500 (US)**: an index that includes 500 of the largest US-listed companies. However, our dataset includes 420 assets.
- **NASDAQ 100 (US)**: an index composed of the 100 largest non-financial companies listed on the NASDAQ exchange, with a strong focus on the technology sector.

4.2 Properties of the problem

From raw prices, we computed simple returns $r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$, which were then used to estimate expected returns and the sample covariance matrix for each dataset. Table 2 shows three useful indicators: the number of assets, the average pairwise correlation of returns and the condition number of the variance-covariance matrix. The first quantifies the size of the investment universe, the second captures the degree of co-movement across assets, and the last provides a measure of numerical stability of the optimization problem.

Dataset	#Assets	Avg correlation	Condition number
FTSE100	82	0.34	356.73
EUROSTOXX50	46	0.47	204.39
DOWJONES	28	0.46	98.27
NASDAQ100	70	0.39	439.05
S&P500	420	0.41	4747.79

Table 2: Summary statistics of the datasets

The table shows moderate correlation (below 0.5) between asset movements, while the condition number varies considerably across datasets.

In particular, the **S&P500** dataset is highly ill-conditioned, which makes it a challenging test case for the algorithms considered. Its inclusion in our analysis allows us to investigate the performance of both FW-based methods and PGD under adverse optimization conditions, thereby complementing the insights obtained from smaller and better-conditioned datasets such as **DOWJONES**.

4.3 The role of the condition number in optimization problems

The condition number of the covariance matrix Σ is known to have a significant impact on the behavior of first-order optimization methods. The condition number κ indicates that the direction of greatest curvature has κ times more curvature than the direction of least curvature. When this quantity is low, the optimization problem is numerically stable, which facilitates faster and smoother convergence. By contrast, a large condition number κ indicates near-collinearities between assets and leads to ill-conditioning [11], resulting in an irregular landscape characterized by both extremely steep and nearly flat directions. As noted in [7], *“the condition number gives an indication of the tightness of the bounds on the curvature of f , and therefore of the difficulty to optimize it: the larger the value of κ is, the slower is the convergence of the algorithm.”*

5 Experimental Results

In this section, we present and analyze the experimental results obtained by applying PGD, FW, AFW and PFW to the described datasets. We compare the convergence behavior, computational efficiency and portfolio allocation related to the sparsity of the solution, highlighting the impact of dataset characteristics such as size and conditioning with relation to each method. Each method is run with the following settings:

- maximum number of iterations: 200
- tolerance parameter $\epsilon \geq 0$: $1e - 5$
- convergence criteria as defined in Sections 3.1.1 and 3.2.1

5.1 Convergence analysis

5.1.1 Frank-Wolfe and variants

The results below clearly demonstrate the advantages of using a line search compared to diminishing stepsize in terms of iterations. In line with the literature, we can see that PFW and AFW with a line search show linear decrease. In contrast, the original FW decreases linearly in favorable conditions, but performs sublinearly in the worst-case scenario, as demonstrated by the **S&P500**, and **NASDAQ100**. More zig-zagging can be observed for all algorithms when using the diminishing stepsize. The results for the Frank-Wolfe method and its main variants highlight the differences

in terms of number of iterations required and, as we will see in Section 7.5, in the corresponding CPU times¹.

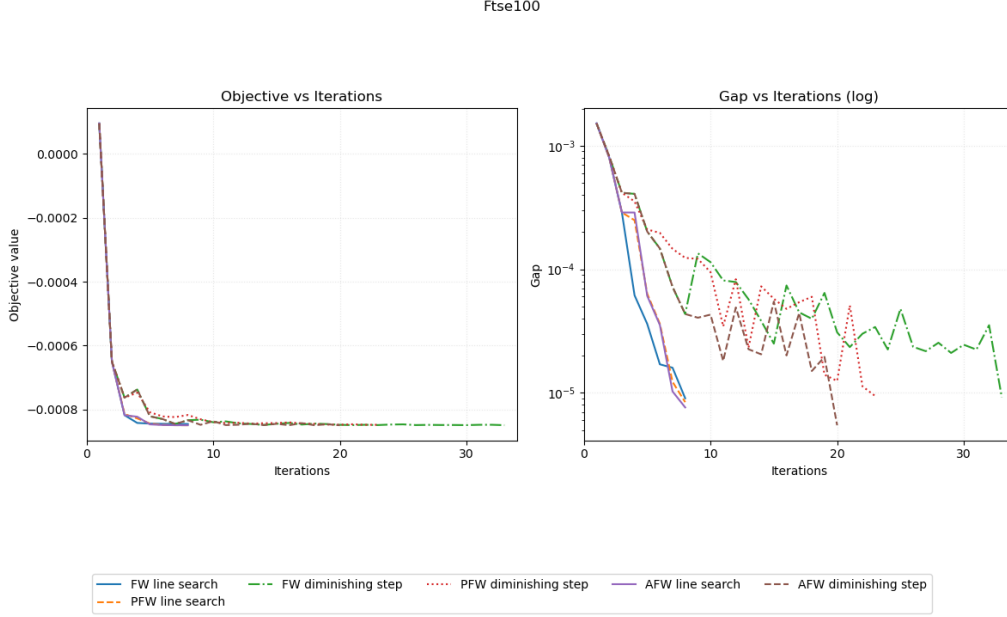


Figure 1: Convergence of FW, AFW and PFW (FTSE100). Objective vs iterations (left) and duality gap (right).

Figure 1 shows the convergence of all FW and its variants exploited on FTSE100. In particular, all FW variants with line search converge in a few iterations, and it is possible to notice a relatively fast linear decrease. The original FW meets the tolerance for the optimality gap in 8 iterations with a duality gap of $\approx 9.0 \times 10^{-6}$, while AFW and PFW also stop in 8 iterations with slightly smaller gaps and objective values. Using diminishing stepsize requires more iterations: AFW requires 20 iterations to reach the tolerance, PFW converges in 23 and FW in 33.

¹Performance might be influenced by initialization: PGD and FW were initialized at a random point on the simplex, while FW variants were initialized on a random vertex, following [9]

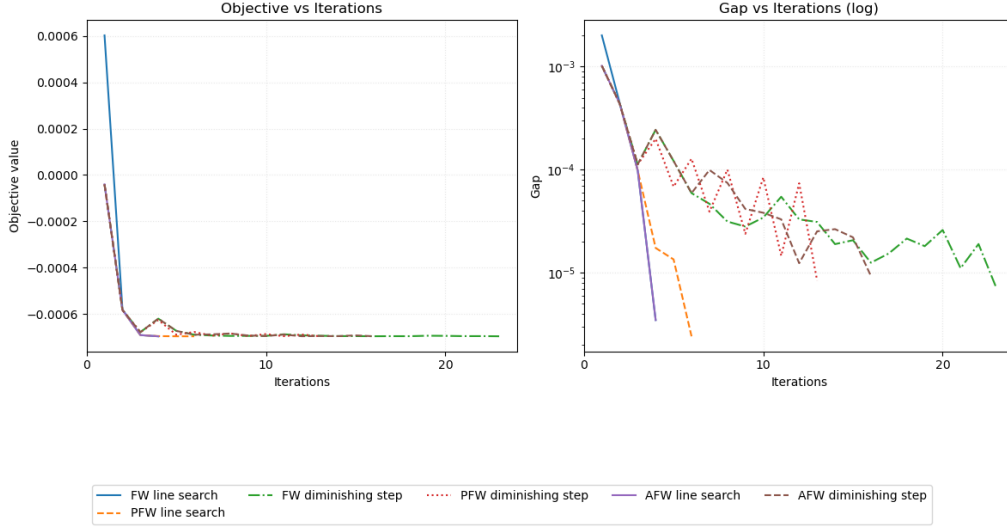


Figure 2: Convergence of FW, AFW and PFW (EUROSTOXX50). Objective vs iterations (left) and duality gap (right).

In EUROSTOXX50 dataset, traditional FW and AFW with line search reach the target tolerance ϵ in only 4 iterations, while PFW with line search requires 6 iterations and achieves an even smaller gap ($\approx 2.4 \times 10^{-6}$). In line with the theory, when adopting diminishing rule, progress takes longer: classic FW requires 23 iterations, AFW requires 16 steps, and PFW 13, typically zig-zagging around gaps of order 10^{-4} before stopping.

DowJones

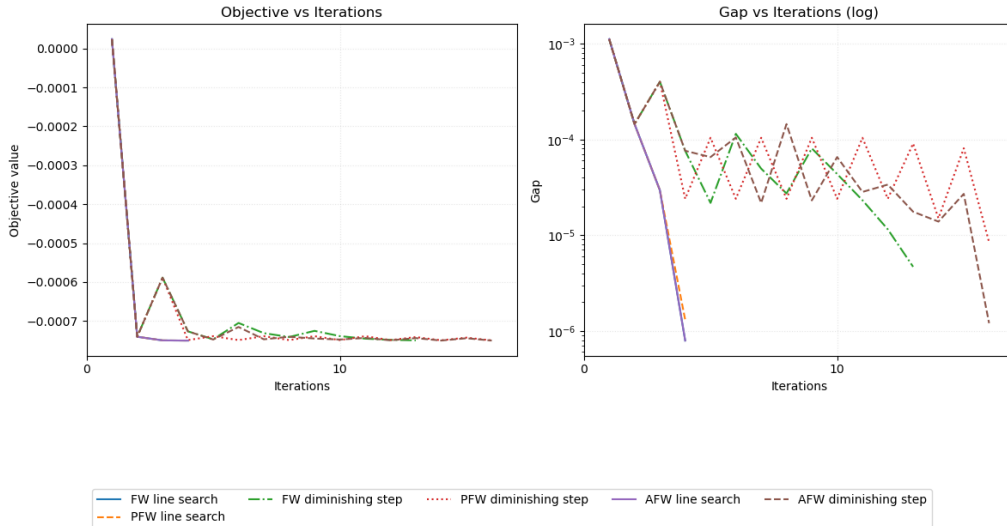


Figure 3: Convergence of FW, AFW and PFW (DOWJONES). Objective vs iterations (left) and duality gap (right).

On the DOWJONES dataset, which is small and well conditioned with respect to the others, FW, PFW and AFW with line search achieve convergence in only 4 iterations. With diminishing stepsize, convergence requires more iterations, but less

if compared with other results: FW stabilizes after 13 iterations, while AFW and PFW require 16 iterations.

SP500

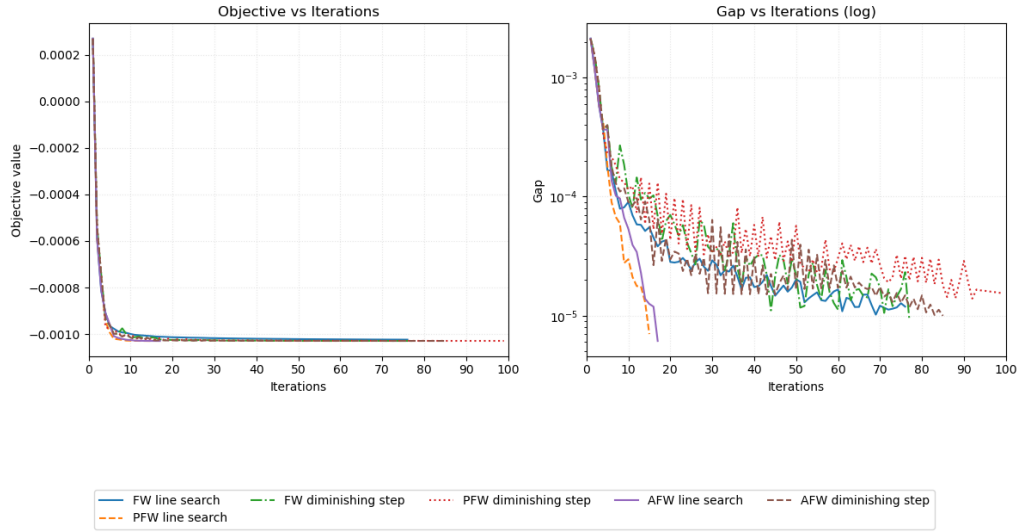


Figure 4: Convergence of FW, AFW and PFW (S&P500). Objective vs iterations (left) and duality gap (right).

The **S&P500** dataset provides a clear example of the challenge posed by ill-conditioning. Among the FW variants with line search, AFW converges in 17 iterations, while PFW performs slightly better, achieving the target in just 15 iterations. By contrast, the FW method with line search is substantially slower here, taking 76 iterations to reach convergence and performing similarly to its diminishing stepsize alternative (77 iterations). The zig-zagging phenomenon is also clearly visible in PFW and AFW variants that exploit diminishing stepsize, which take 99 and 85 iterations respectively, showing irregular trajectories of the duality gap.

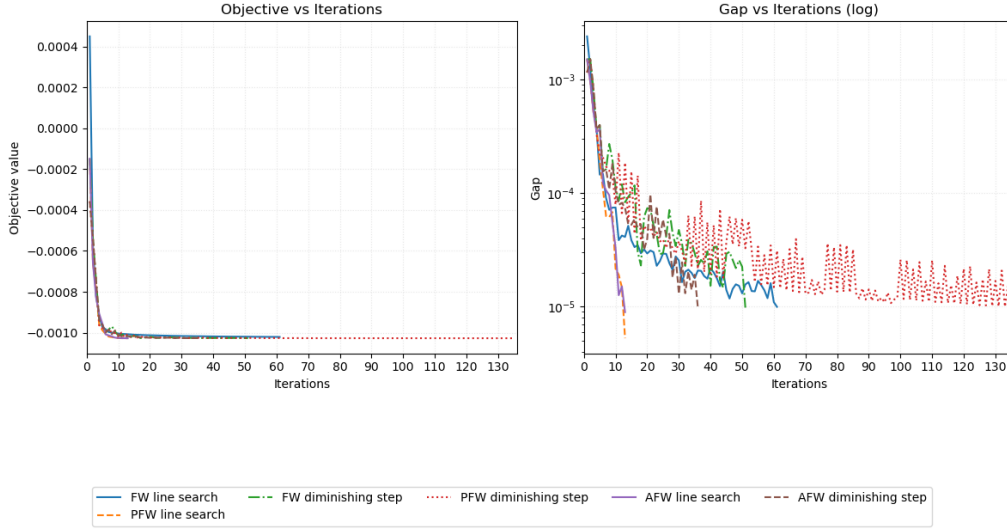


Figure 5: Convergence of FW, AFW and PFW (NASDAQ100). Objective vs iterations (left) and duality gap (right).

On NASDAQ100 dataset, with respect of the algorithms with line search, PFW performs the best, converging in 13 steps with a final gap of $5.2732e - 06$, slightly smaller than the one that achieves AFW which also requires 13 iterations. FW algorithm, on the other hand, is substantially slower to converge, reaching tolerance after 61 iterations. With diminishing stepsizes, AFW converges in 36 iterations, FW in 51, while PFW deteriorates and requires 135 iterations.

5.1.2 Projected Gradient Descent

For our experiments, the results achieved through PGD-based methods are summarized in Table 3, while a graphical visualization can be found in Appendix 7.2.

Dataset	PGD variant	Final f	Norm of gradient mapping	# Iterations
FTSE100	Fixed	-0.00084887	0.000029573	193
FTSE100	Exact	-0.00084887	0.000029573	193
EUROSTOXX50	Fixed	-0.0006972	0.000028206	193
EUROSTOXX50	Exact	-0.0006972	0.000028206	193
DOWJONES	Fixed	-0.00075042	0.000014080	150
DOWJONES	Exact	-0.00075042	0.000014080	150
S&P500	Fixed	-0.00102650	0.000466000	200
S&P500	Exact	-0.00102650	0.000466000	200
NASDAQ100	Fixed	-0.00102720	0.000046982	196
NASDAQ100	Exact	-0.00102720	0.000046982	196

Table 3: PGD results by datasets: final objective value f , gradient mapping’s norm, and iteration count.

From the table, we can observe that for all datasets, PGD with fixed step and PGD with exact line produce identical trajectories achieving the same final f , $\|g_c(x_k)\|$, and the same iteration counts. This happens because both algorithms consistently select a stepsize of 1 at every iteration, in all tested datasets.

We explain this behavior by noticing that for all datasets the largest eigenvalue of the covariance matrix is well below 1 resulting in values of γ greater than 1 when computed by both fixed step ($\gamma = \frac{1}{L}$) and line search ($\gamma = -\frac{\nabla f(x)^\top d}{2\eta d^\top \Sigma d}$). However, γ is bounded between 0 and 1 to ensure feasibility, so the algorithms will consistently select $\gamma = 1$, as it is the maximum allowed value.

In practice, the fixed step variant is preferable for our problem because it avoids the computational burden imposed by the line search.

Looking at the convergence results, we can notice that the algorithms stop at 193 iterations for FTSE100 and EUROSTOXX50, at 196 for NASDAQ100, and at 150 for DOWJONES. For all cases, the final $\|g_c(x_k)\|$ remains above the tolerance ϵ . The algorithms stop due to insufficient decrease of the optimality gap. In S&P500 dataset the algorithms stop because they reach the maximum number of iterations allowed.

In general, PGD performs significantly worse than FW and its variants. This is evident in terms of both the number of iterations and the value of the optimality condition, as well as the amount by which the objective function decreases with each iteration (see the appendix 7.3 for visualization). These results are not surprising, given the well-known poor performance of (projected) gradient methods in ill-conditioned problems [1].

5.1.3 CPU-time comparisons

This section compares the computational cost of the algorithms considered. The analysis takes into account both the number of iterations and the computational time required to reach convergence.

The plots (see Appendix 7.5) clearly highlight a separation between the Frank-Wolfe methods (including the variants) and the Projected Gradient method. The former consistently requires fewer iterations and less computational time, compared to the latter. The gap between the two families of methods becomes more pronounced as the dataset size and condition number grow, suggesting that problem complexity accentuates these computational differences.

With the exception of the S&P500, the CPU time for all FW algorithms is approximately the same and very small within datasets. This is a consequence of the small number of variables in our datasets, which makes it difficult to distinguish algorithmic efficiency from fluctuations in CPU time caused by spurious factors such as background processes running on the machine.

More interesting insights can be gained from our biggest dataset, where PFW and AFW with line search perform three to five times better than their diminishing stepsize counterpart (6.3ms vs 32.5ms for PFW, and 7.3ms vs 21.9ms for AFW). The two algorithms implementing classical FW perform in between the AFW algorithms. Both PGD algorithms require significantly more time: the fastest PGD variant requires about eight times the time required by the fastest PFW algorithm(6.3ms for PFW-line-search vs 48.6ms for PGD-diminishing).

A clear difference between the PGD algorithms is also visible. Considering that the PGD variants follow the same trajectories, the algorithm with the fixed stepsize should be preferred because it is not burdened by the computation of the line search and requires lower CPU time across datasets. The difference between variants also increases with the size of the dataset.

5.2 Sparsity

As with CPU, sparsity barely varies across algorithms for low-dimensional, better-conditioned datasets. Some differences are visible for NASDAQ100 and S&P500.

Firstly, PGD tends to select more assets, particularly in the second dataset. The difference may seem small (13 assets versus 9 for algorithms with sparser solutions), but it can be significant for investors who don't want their portfolio to vary too much. Secondly, the sparsest solutions are obtained with PFW and AFW using diminishing stepsizes, which are the FW-family algorithms that exhibit higher duality gaps. This again aligns with the theory from [8] that we reported above.

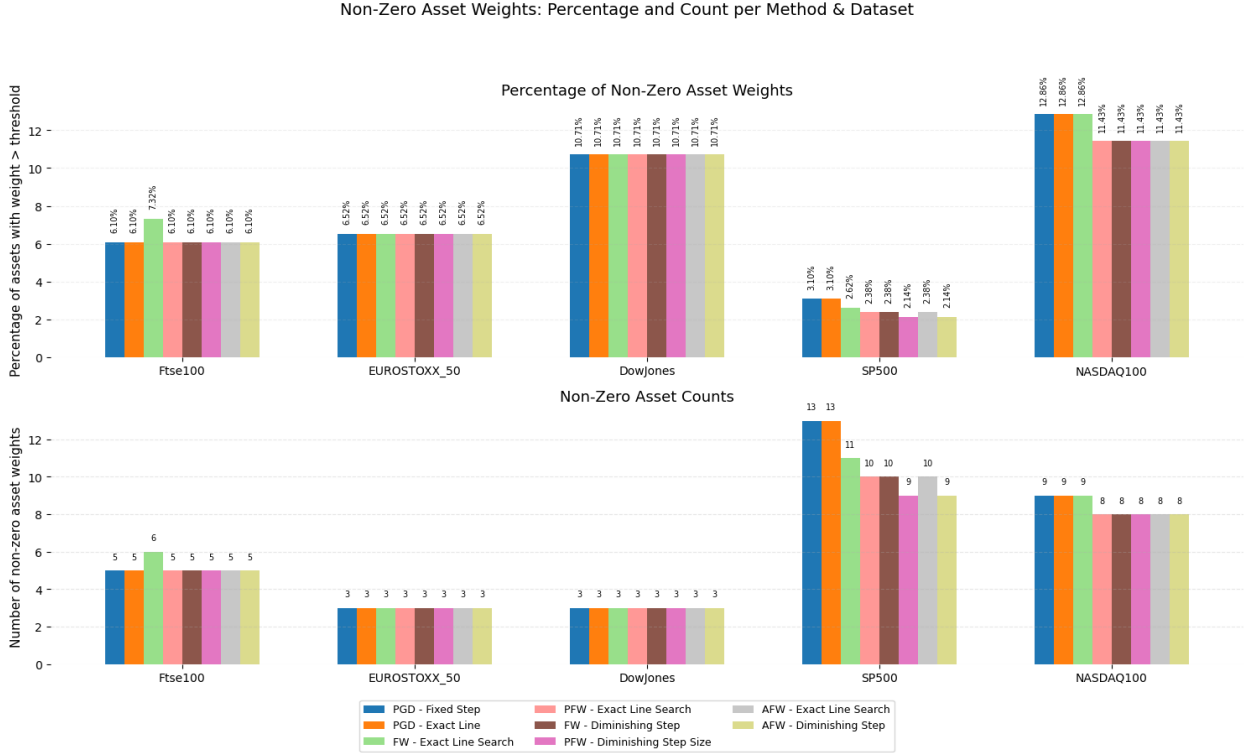


Figure 6: Sparsity of final portfolios by dataset and method. Top: percentage of strictly-positive weights; bottom: count of positive weights.

Although we deliberately refrain from drawing economic conclusions given our lack of domain expertise, it is worth noting that stronger sparsity, as obtained by AFW and PFW, could potentially translate into more concentrated allocations on a few relevant portfolio assets, a property that in practice may be interesting for interpretability and portfolio management.

6 Conclusion

PFW and AFW with line search have the best results, converging linearly with few iterations in all datasets. They produce sparse results and require the least CPU time in almost every problem. Their supremacy over the other algorithms is particularly evident in our larger, worse-conditioned datasets. For our problem, their performance is equivalent, and none is visibly better than the other.

FW with line search performs similarly to the above described algorithms for the FTSE100, EUROSTOXX50, and DOWJONES datasets, but suffers from sublinear con-

vergence in the remaining datasets. Its CPU time remains low for all problems, but the sparsity was among the lowest on average.

On the other hand, all diminishing variants perform sublinearly across datasets and zig-zagging is evident. They require similar computational time as the above algorithms, except for **S&P500**, where they need more time. PFW is particularly costly for the latter dataset and for **NASDAQ100**. Concerning sparsity, PFW and AFW with this stepsize strategy have the sparser results on average.

Finally, PGD algorithm performs worse than all others, never reaching the pre-defined convergence tolerance. Repeated projections make it the least competitive option when the dimension grows or conditioning deteriorates. Both variants select the highest number of assets on **S&P500** and **NASDAQ100** and have a significantly longer computational time in all datasets. The only difference between the stepsize strategies is in CPU time, where line search appears more costly without providing any additional benefit.

In light of these facts, the recommended default for this and other similar problems that show nice structure and allow an easy computation of the line search, is to use AFW or PFW. However, we recognize that PGD is very easy to implement and to be explained to decision makers, when dealing with decisions on real-world data, thus remaining a valuable option.

While these results provide interesting insights into the computational performance of the algorithms, we should remember that 420 assets may still be few for a data science problem. Better conclusions would be seen with larger datasets.

A natural next step would be further evaluating how the computational gains translate into practical gains in real life portfolio management. Other interesting analysis would be addressing ill-conditioning first, applying the algorithms to bigger datasets or applying them to different minimization problems.

7 Appendix

7.1 Derivation of the exact line search.

Let $d \in \mathbb{R}^n$ denote a feasible descent direction and consider the step

$$x(\gamma) := x + \gamma d, \quad \gamma \in [0, 1].$$

The objective function is

$$f(x) = \eta x^\top \Sigma x - \bar{r}^\top x,$$

so along the search direction, we define

$$\phi(\gamma) := f(x + \gamma d) = \eta (x + \gamma d)^\top \Sigma (x + \gamma d) - \bar{r}^\top (x + \gamma d).$$

Expanding the quadratic term:

$$(x + \gamma d)^\top \Sigma (x + \gamma d) = x^\top \Sigma x + 2\gamma d^\top \Sigma x + \gamma^2 d^\top \Sigma d,$$

we obtain

$$\phi(\gamma) = \eta (x^\top \Sigma x + 2\gamma d^\top \Sigma x + \gamma^2 d^\top \Sigma d) - \bar{r}^\top x - \gamma \bar{r}^\top d.$$

Differentiating with respect to γ :

$$\phi'(\gamma) = 2\eta d^\top \Sigma x + 2\eta \gamma d^\top \Sigma d - \bar{r}^\top d.$$

Since $\nabla f(x) = 2\eta \Sigma x - \bar{r}$, this can be rewritten as

$$\phi'(\gamma) = \nabla f(x)^\top d + 2\eta \gamma d^\top \Sigma d.$$

Setting $\phi'(\gamma) = 0$ gives the unconstrained minimizer:

$$\gamma^* = -\frac{\nabla f(x)^\top d}{2\eta d^\top \Sigma d}. \quad (10)$$

The exact line search then selects

$$\gamma^* = \arg \min_{\gamma \in [0, \gamma_{max}]} \phi(\gamma) = \min\{\gamma, \gamma_{max}\},$$

ensuring feasibility of the update.

7.2 Convergence Plots

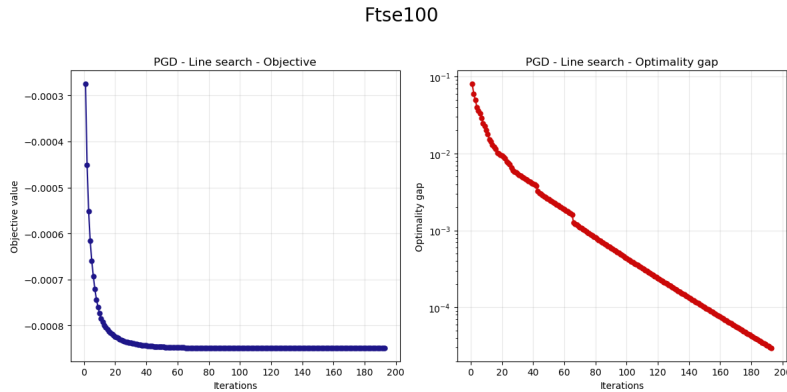


Figure 7: Convergence for PGD-exact line on FTSE100(left objective, right gradient mapping norm)

EUROSTOXX_50

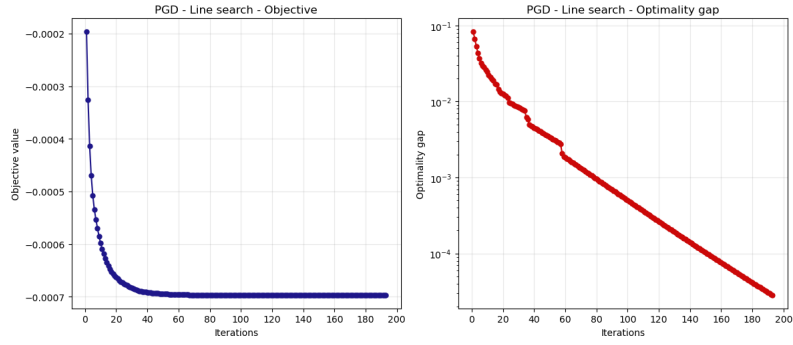


Figure 8: Convergence for PGD-line search on EUROSTOXX50 (left objective, right gradient mapping norm)

DowJones

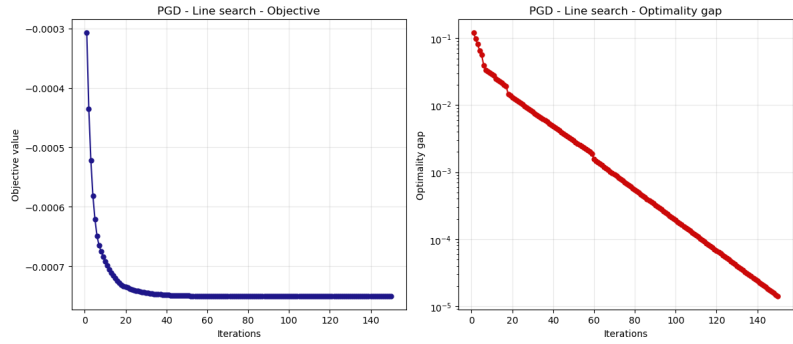


Figure 9: Convergence for PGD-line search on DOWJONES (left objective, right gradient mapping norm)

SP500

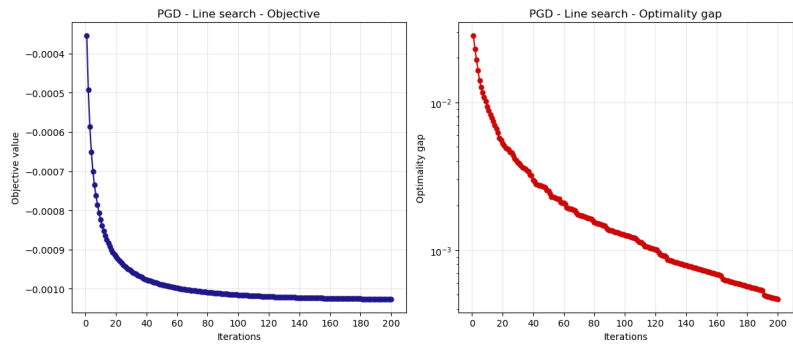


Figure 10: Convergence for PGD-line search on S&P500 (left objective, right gradient mapping norm)

NASDAQ100

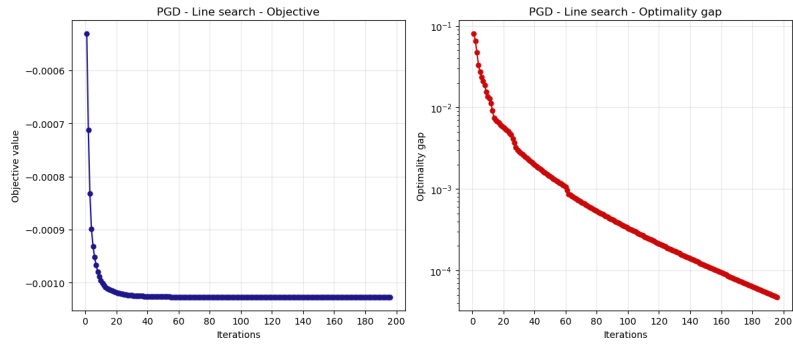


Figure 11: Convergence for PGD-line search on NASDAQ100 (left objective, right gradient mapping norm)

Ftse100

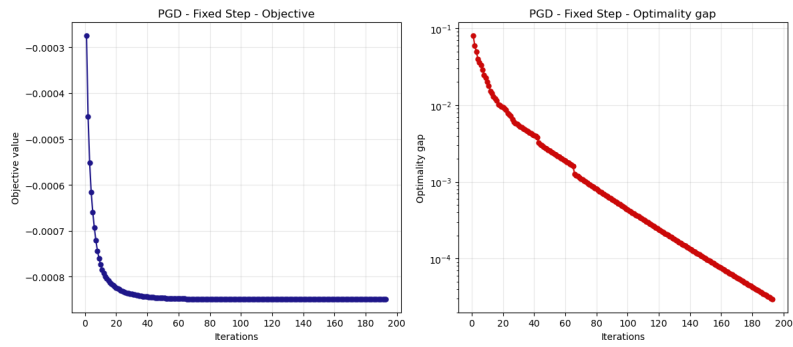


Figure 12: Convergence for PGD-fixed line on FTSE100 (left objective, right gradient mapping norm)

EUROSTOXX_50

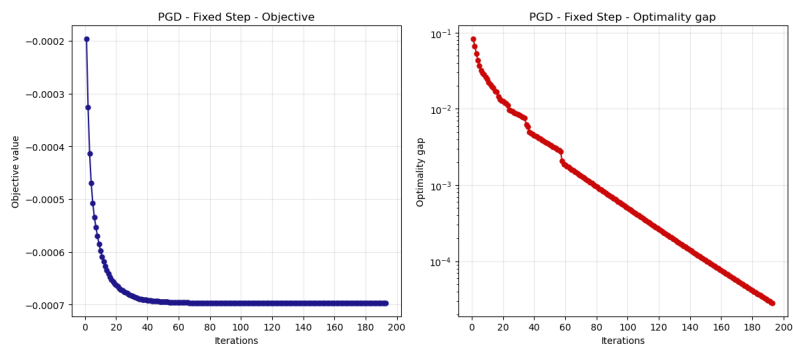


Figure 13: Convergence for PGD-fixed line on EUROSTOXX50 (left objective, right gradient mapping norm)

DowJones

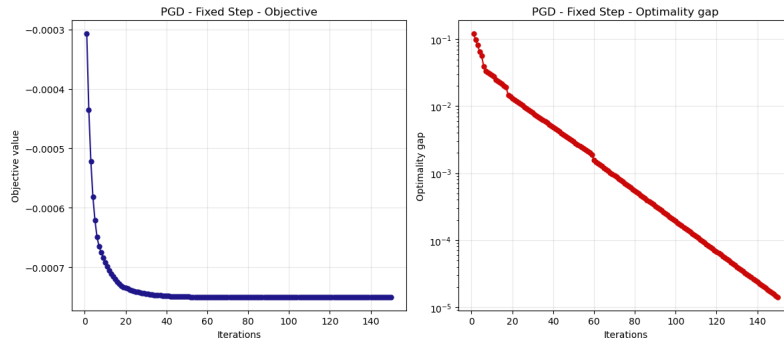


Figure 14: Convergence for PGD-fixed line on DOWJONES (left objective, right gradient mapping norm)

SP500

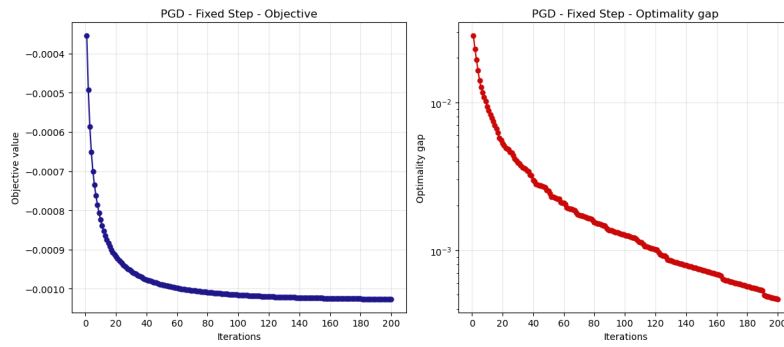


Figure 15: Convergence for PGD-fixed line on S&P500 (left objective, right gradient mapping norm)

NASDAQ100

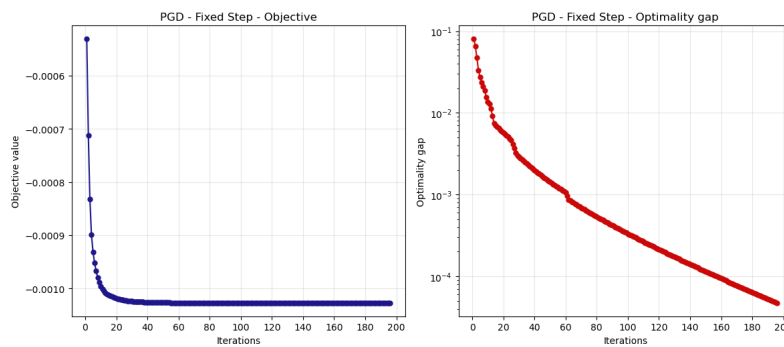


Figure 16: Convergence for PGD-fixed line on NASDAQ100 (left objective, right gradient mapping norm)

Ftse100

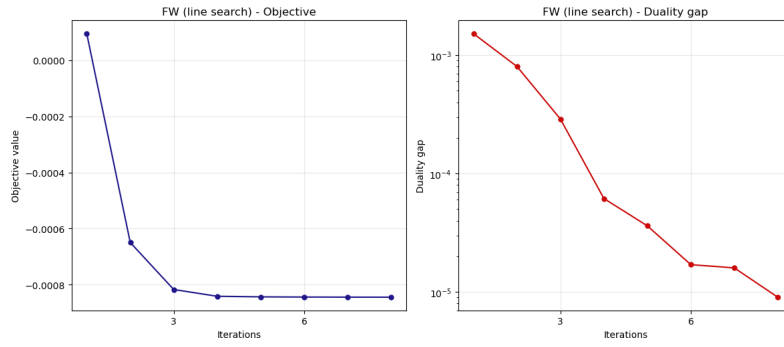


Figure 17: Convergence for FW-line search on FTSE100 (left objective, right duality gap)

EUROSTOXX_50

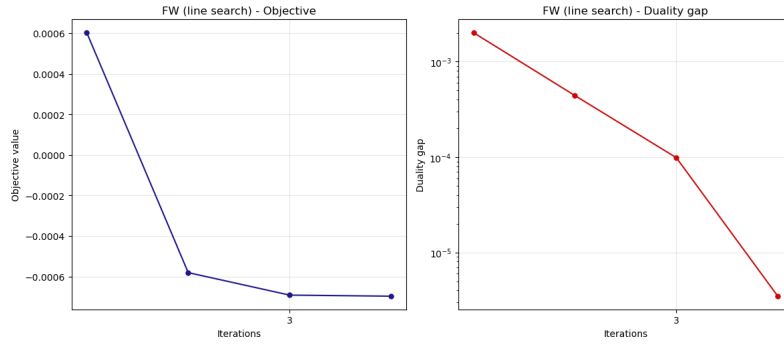


Figure 18: Convergence for FW-line search on EUROSTOXX50 (left objective, right duality gap)

DowJones

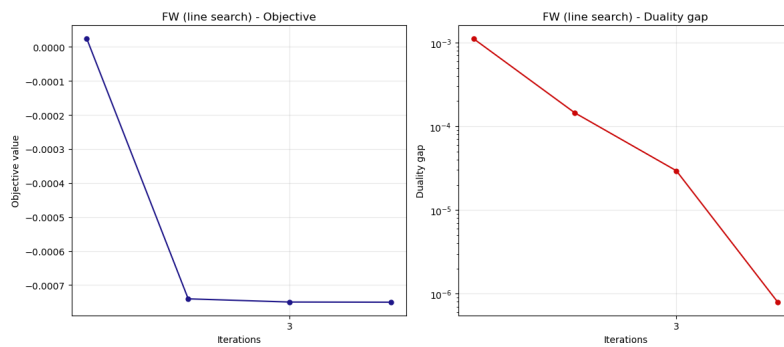


Figure 19: Convergence for FW-line search on DOWJONES(left objective, right duality gap)

SP500

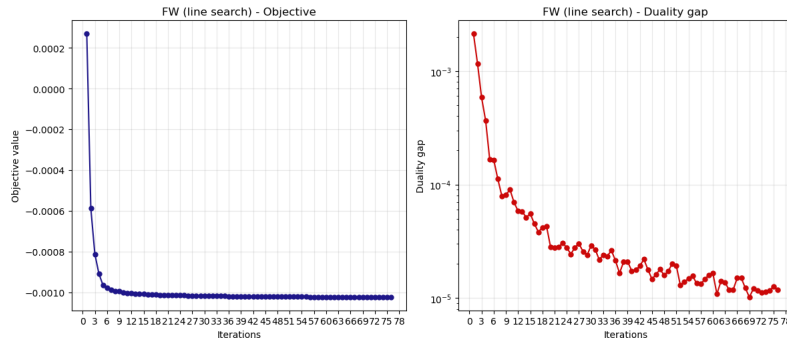


Figure 20: Convergence for FW-line search on S&P500 (left objective, right duality gap)

NASDAQ100

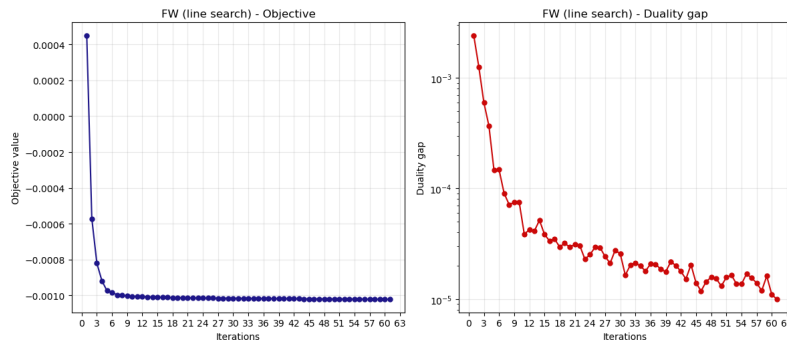


Figure 21: Convergence for FW-line search on NASDAQ100 (left objective, right duality gap)

Ftse100

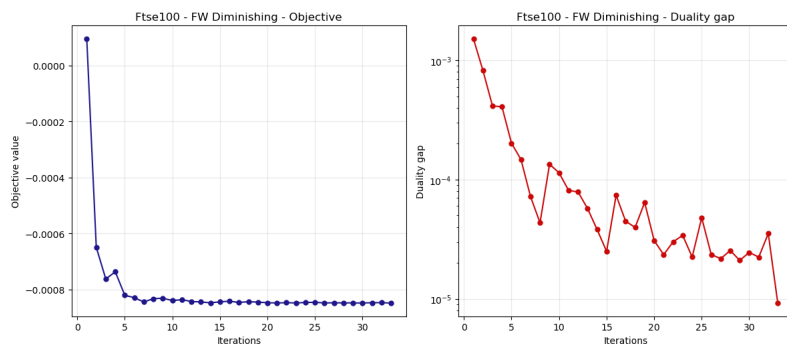


Figure 22: Convergence for FW-Diminishing on FTSE100 (left objective, right duality gap)

EUROSTOXX_50

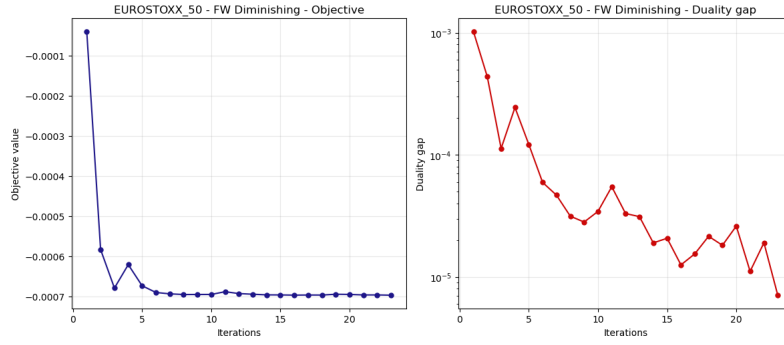


Figure 23: Convergence for FW-Diminishing on EUROSTOXX50 (left objective, right duality gap)

DowJones

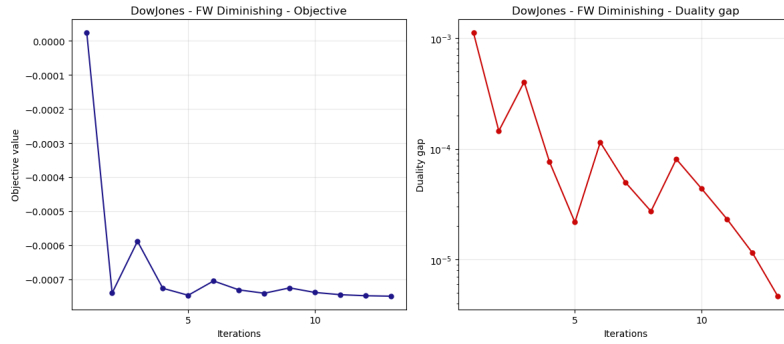


Figure 24: Convergence for FW-Diminishing on DOWJONES (left objective, right duality gap)

SP500

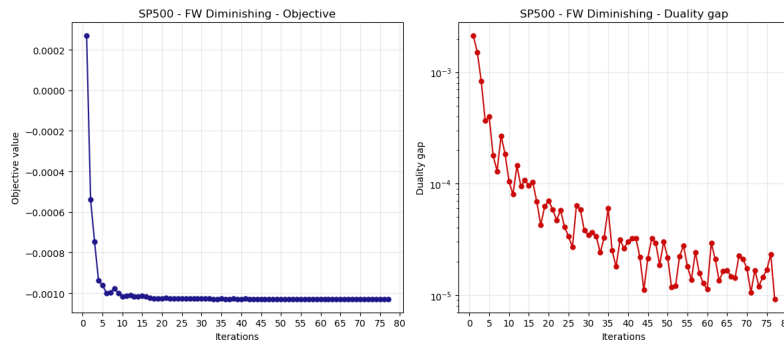


Figure 25: Convergence for FW-Diminishing on S&P500 (left objective, right duality gap)

NASDAQ100

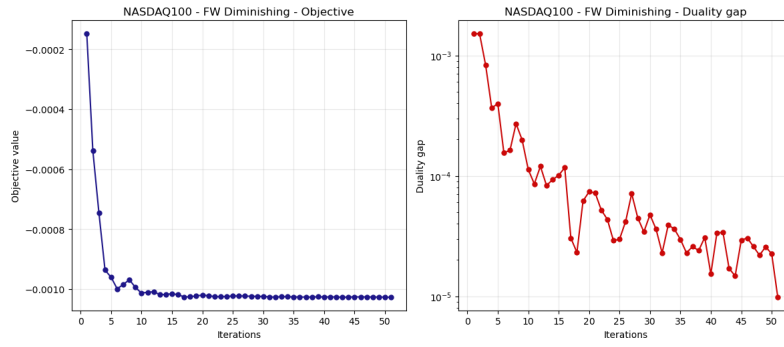


Figure 26: Convergence for FW-Diminishing step on NASDAQ100 (left objective, right duality gap)

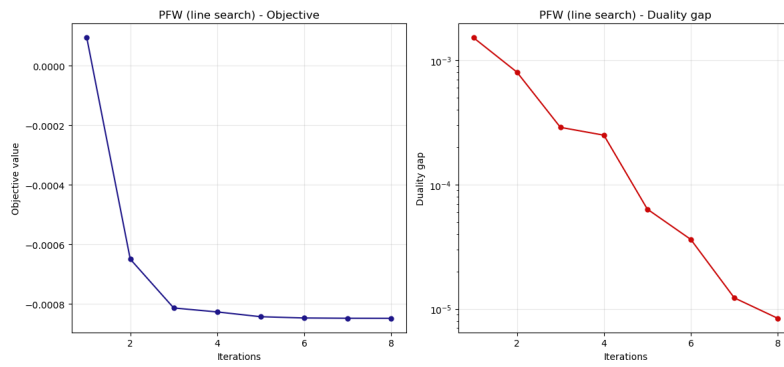


Figure 27: Convergence for PFW-line search on FTSE100 (left objective, right duality gap)

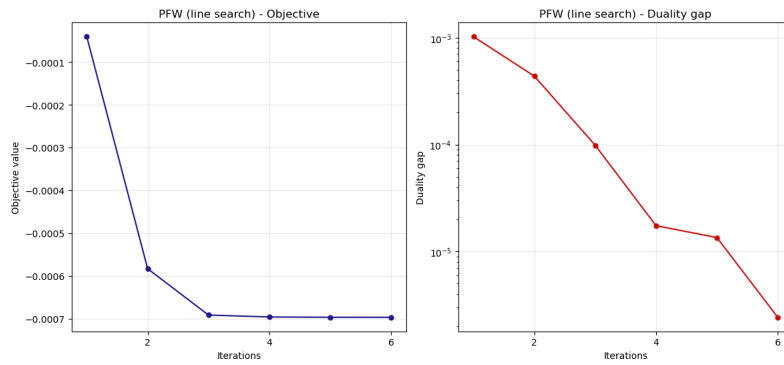


Figure 28: Convergence for PFW-line search on EUROSTOXX50 (left objective, right duality gap)

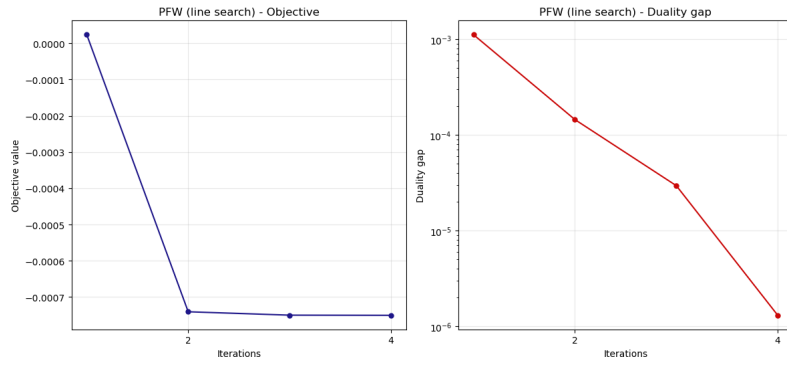


Figure 29: Convergence for PFW-line search on DOWJONES (left objective, right duality gap)

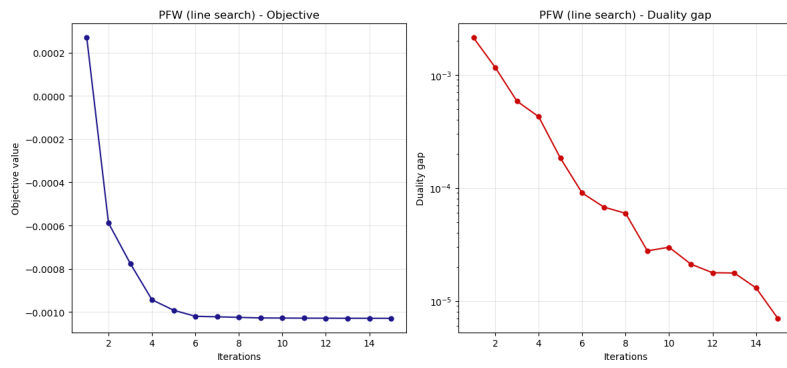


Figure 30: Convergence for PFW-line search on S&P500 (left objective, right duality gap)

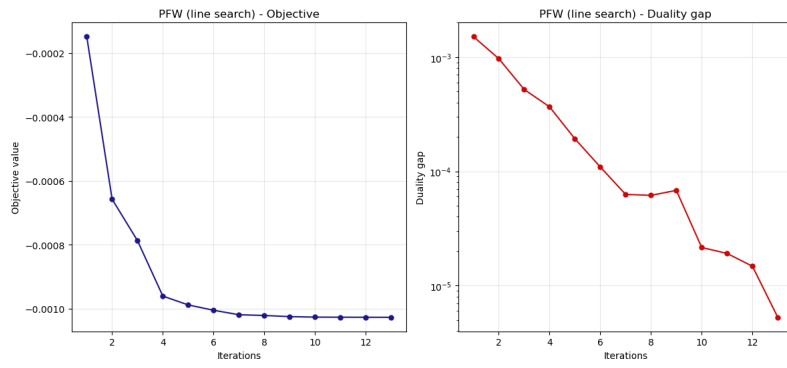


Figure 31: Convergence for PFW-line search on NASDAQ100 (left objective, right duality gap)

Ftse100

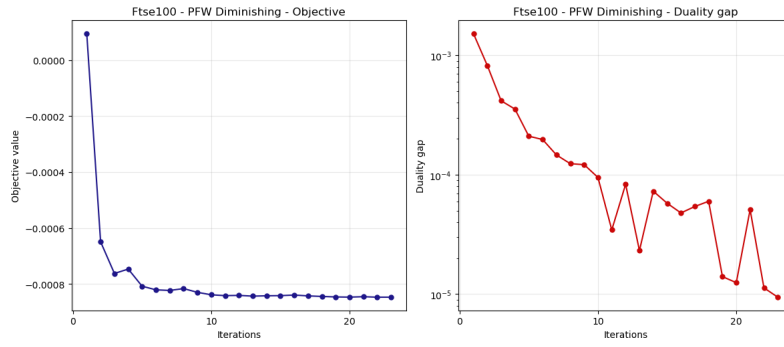


Figure 32: Convergence for PFW-Diminishing step on FTSE100 (left objective, right duality gap)

EUROSTOXX_50

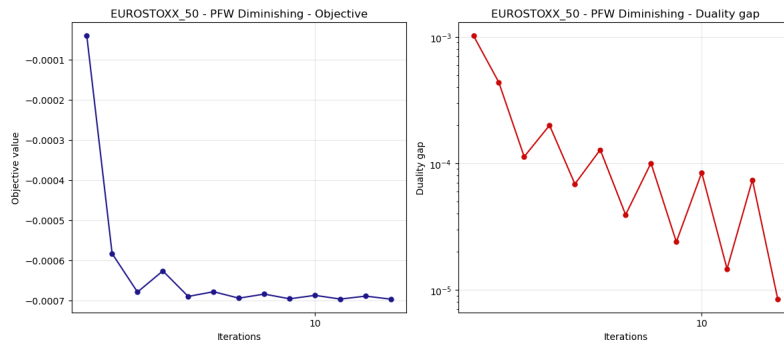


Figure 33: Convergence for PFW-Diminishing step on EUROSTOXX50 (left objective, right duality gap)

DowJones

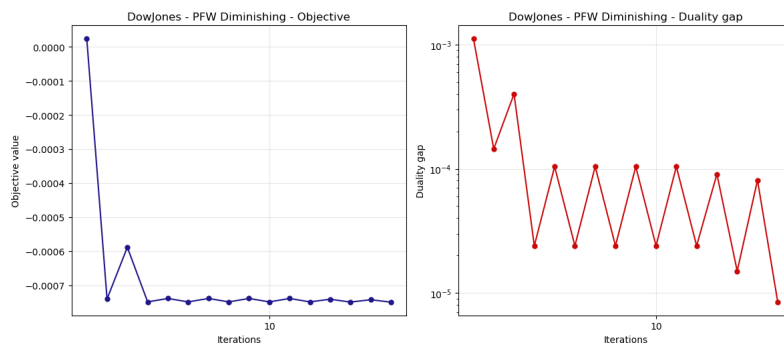


Figure 34: Convergence for PFW-Diminishing step on DOWJONES (left objective, right duality gap)

SP500

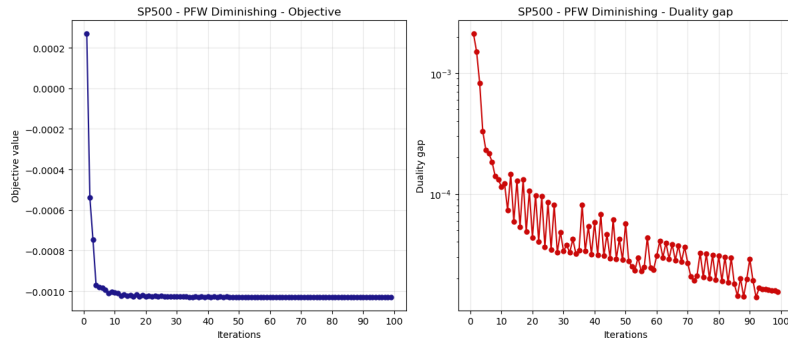


Figure 35: Convergence for PFW-Diminishing step on S&P500 (left objective, right duality gap)

NASDAQ100

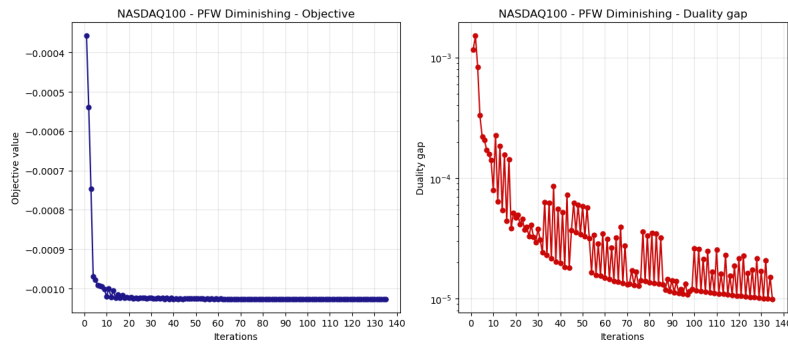


Figure 36: Convergence for PFW-Diminishing step on NASDAQ100 (left objective, right duality gap)

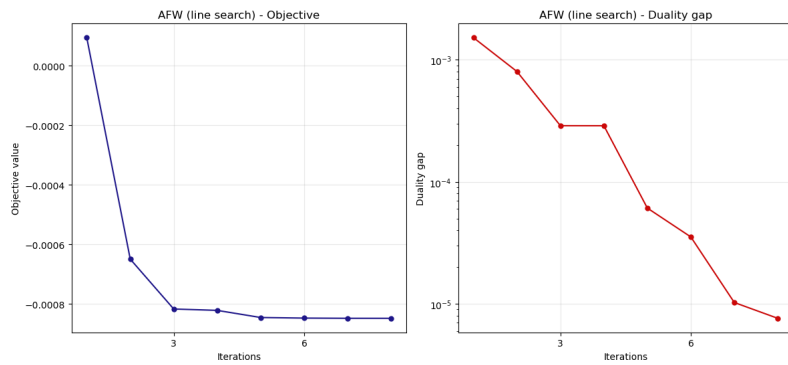


Figure 37: Convergence for AFW-line search on FTSE100 (left objective, right duality gap)

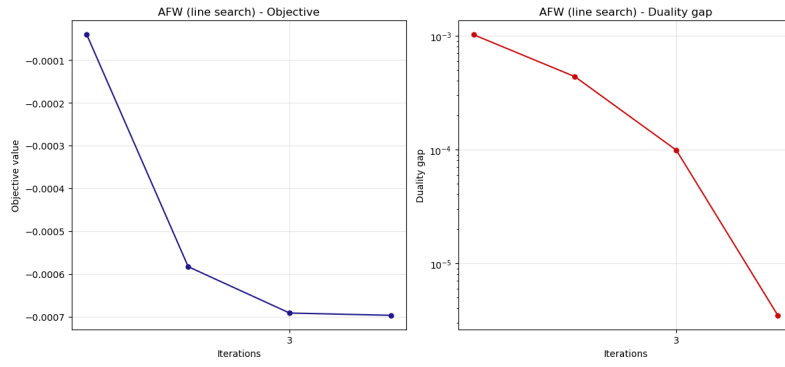


Figure 38: Convergence for AFW-line search on EUROSTOXX50 (left objective, right duality gap)

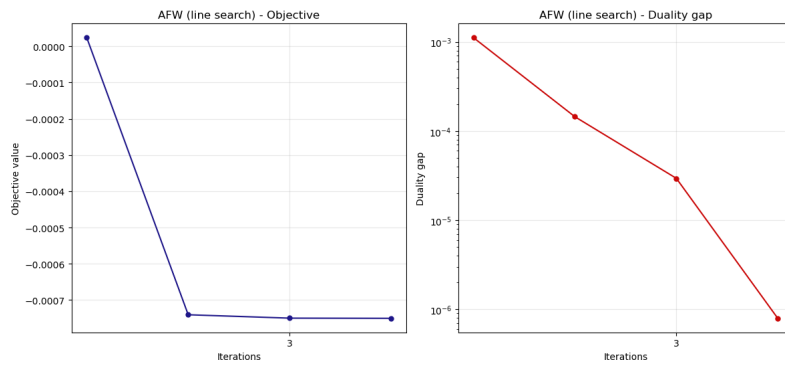


Figure 39: Convergence for AFW-line search on DOWJONES (left objective, right duality gap)

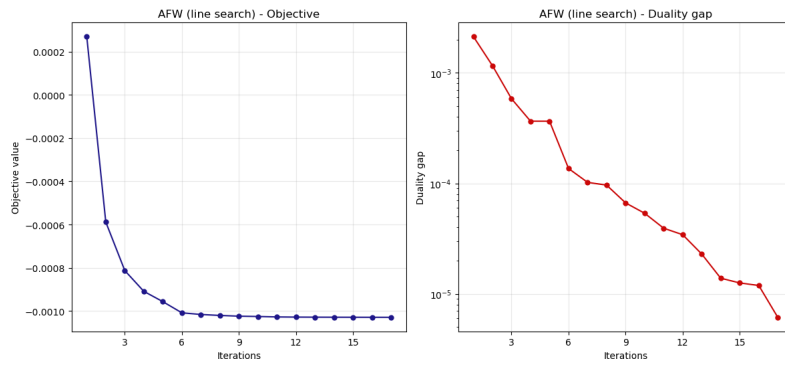


Figure 40: Convergence for AFW-line search on S&P500 (left objective, right duality gap)

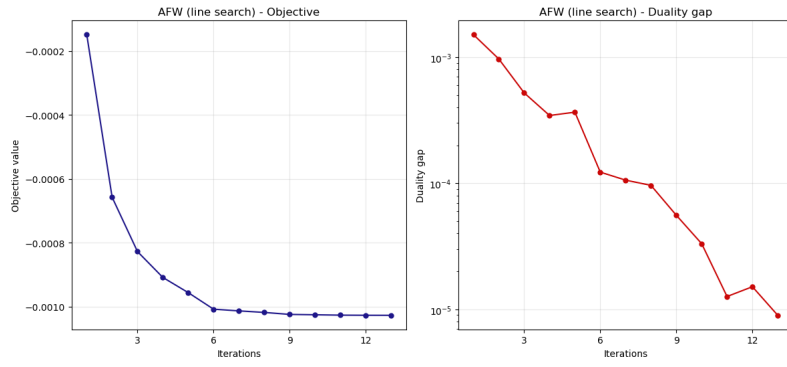


Figure 41: Convergence for AFW-line search on NASDAQ100 (left objective, right duality gap)

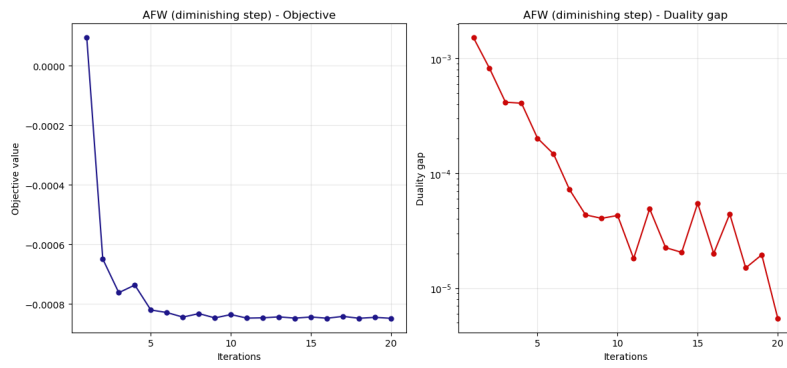


Figure 42: Convergence for AFW-Diminishing step on FTSE100 (left objective, right duality gap)

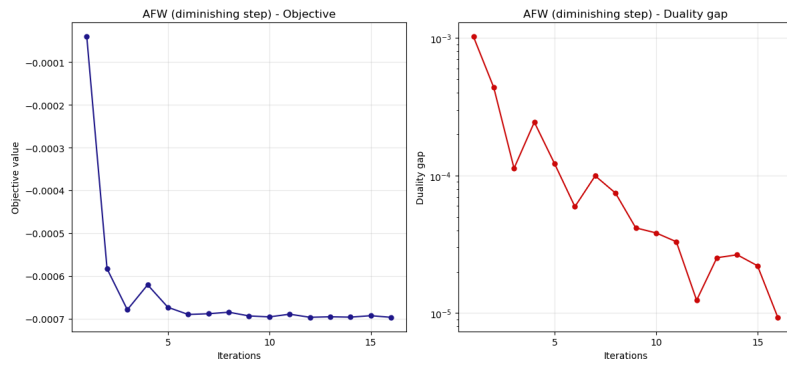


Figure 43: Convergence for AFW-Diminishing step on EUROST0XX50 (left objective, right duality gap)

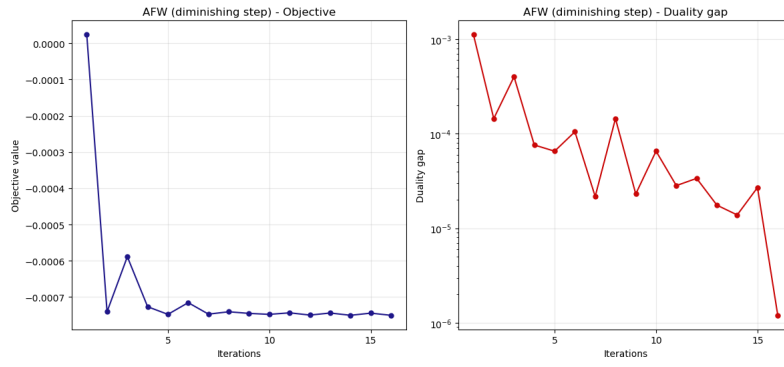


Figure 44: Convergence for AFW-Diminishing step on DOWJONES (left objective, right duality gap)

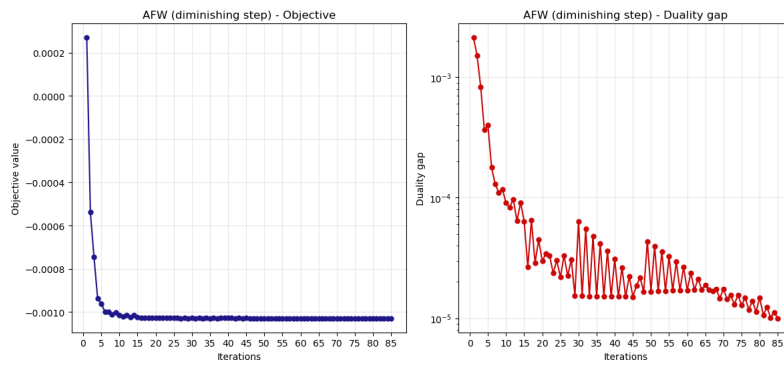


Figure 45: Convergence for AFW-Diminishing step on S&P500 (left objective, right duality gap)

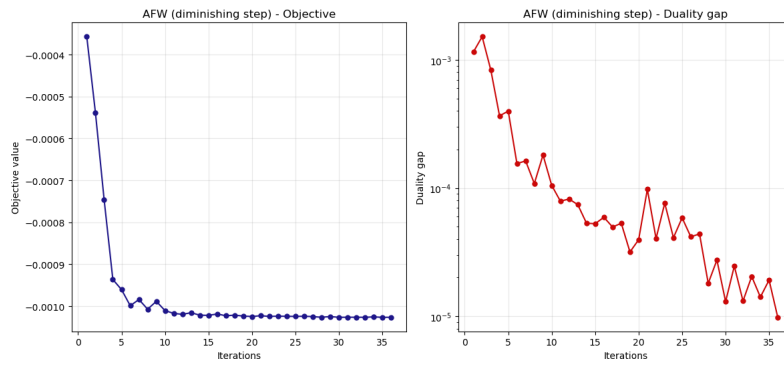


Figure 46: Convergence for AFW-Diminishing step on NASDAQ100 (left objective, right duality gap)

7.3 Convergence of all methods

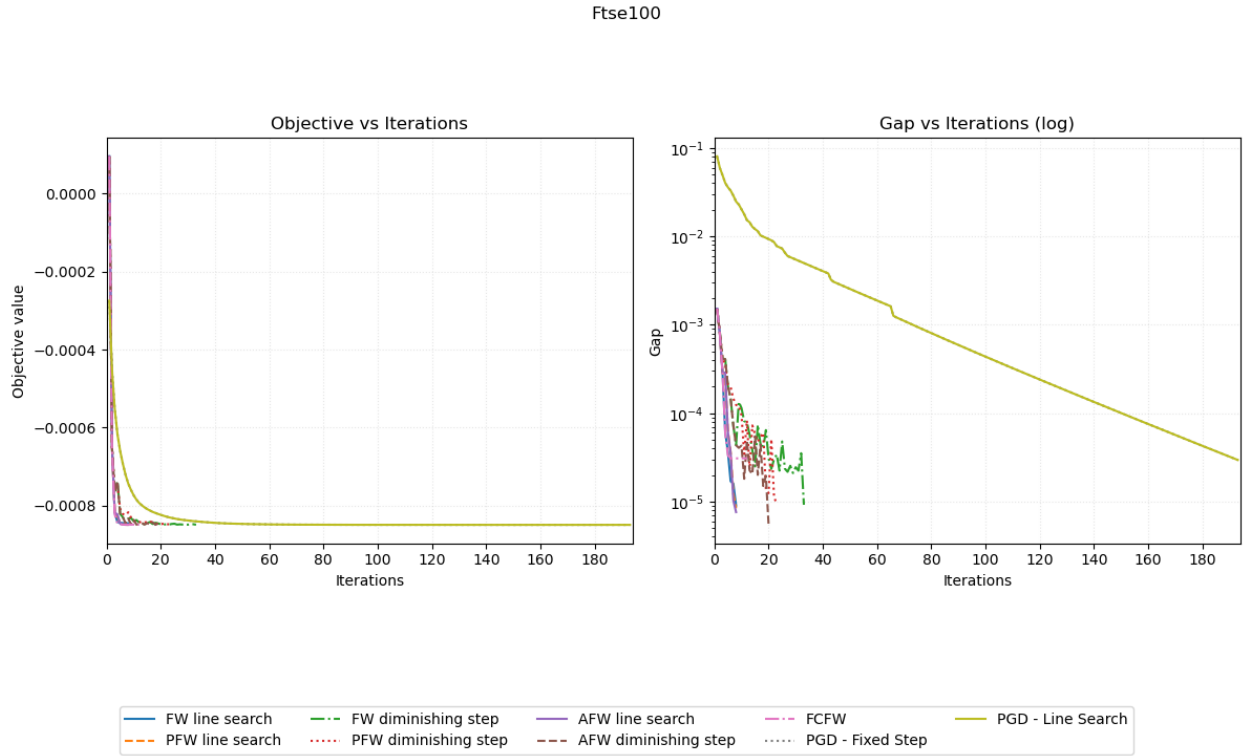


Figure 47: Convergence of all methods on FTSE100: left objective, right optimality gap

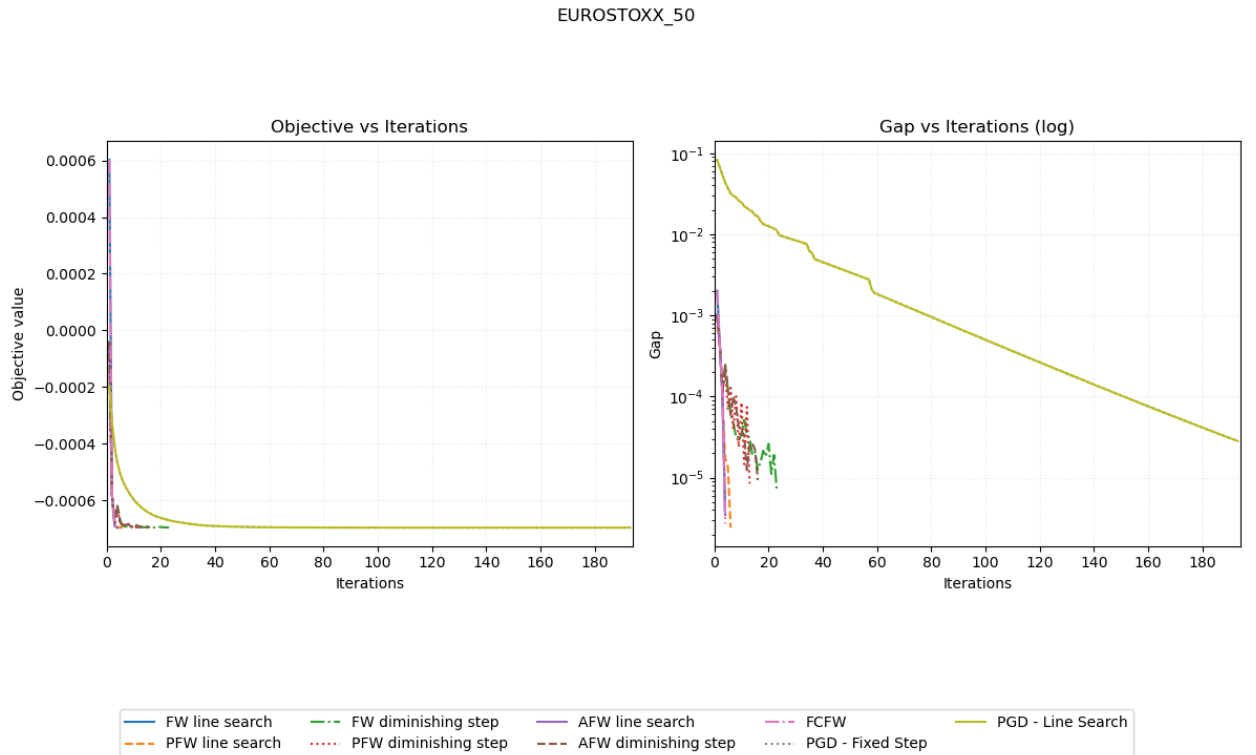


Figure 48: Convergence of all methods on EUROSTOXX50: left objective, right optimality gap

DowJones

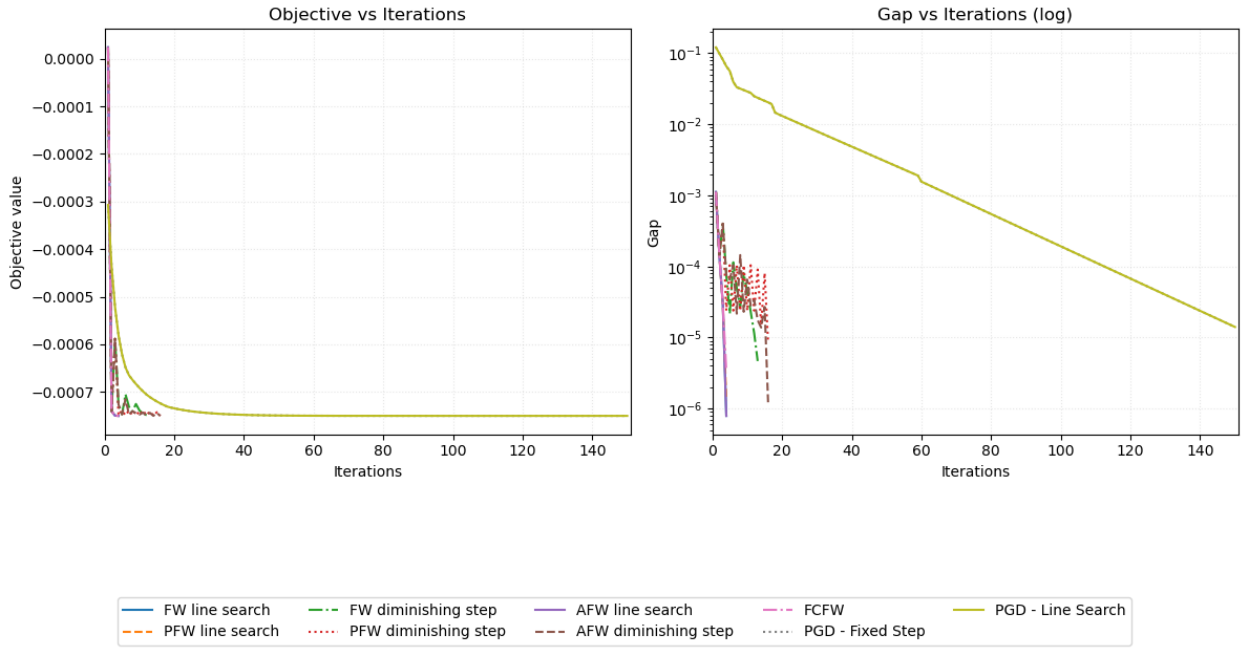


Figure 49: Convergence of all methods on DOWJONES: left objective, right optimality gap

SP500

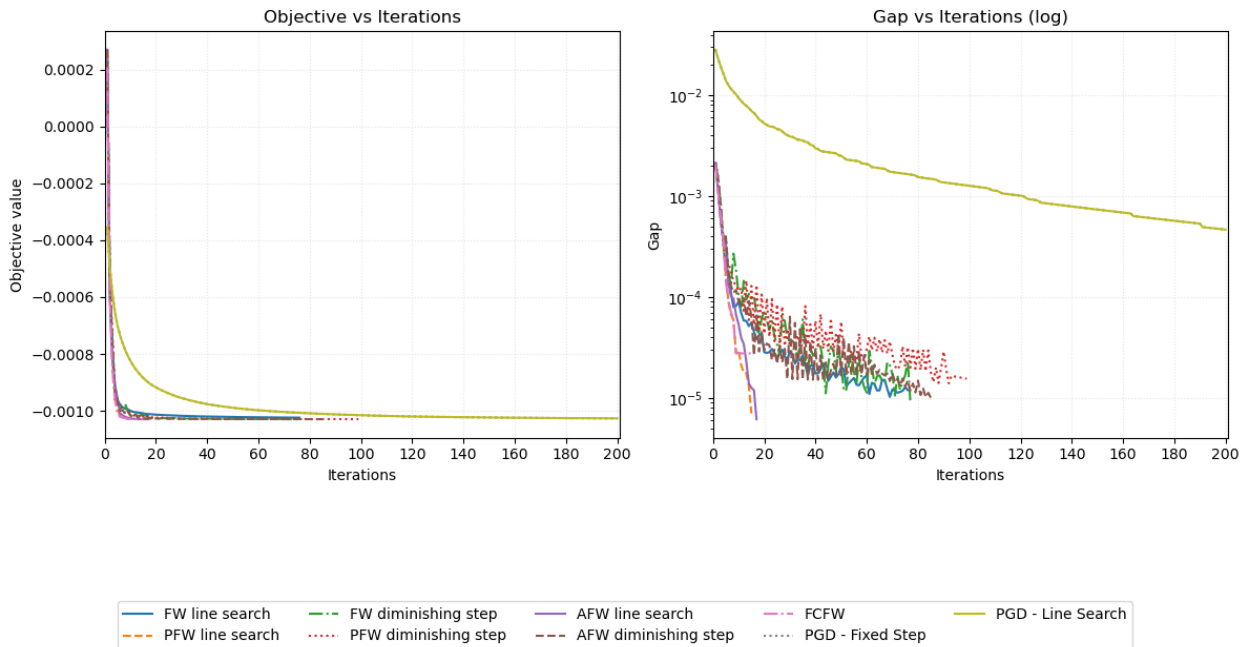


Figure 50: Convergence of all methods on S&P500: left objective, right optimality gap

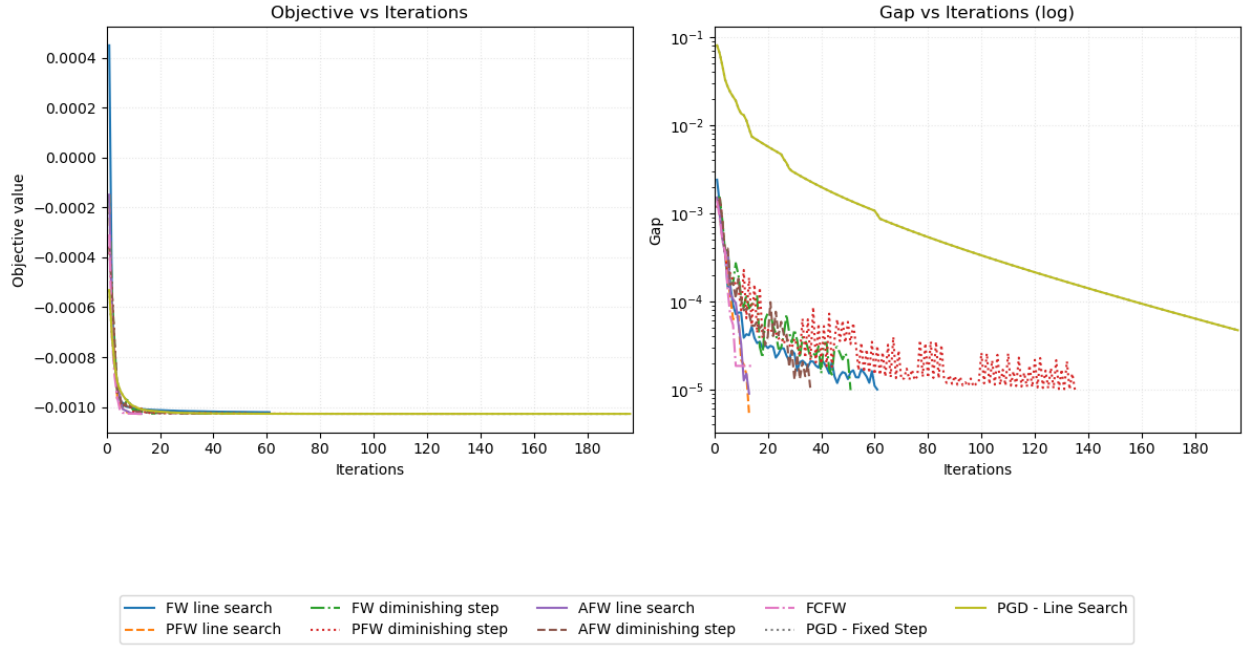


Figure 51: Convergence of all methods on NASDAQ100: left objective, right optimality gap

7.4 Stepsizes plots

Note: Here, we only included the stepsize trajectories of the FW methods. The PGD stepsizes were equal to 1 for both algorithms throughout all iterations.

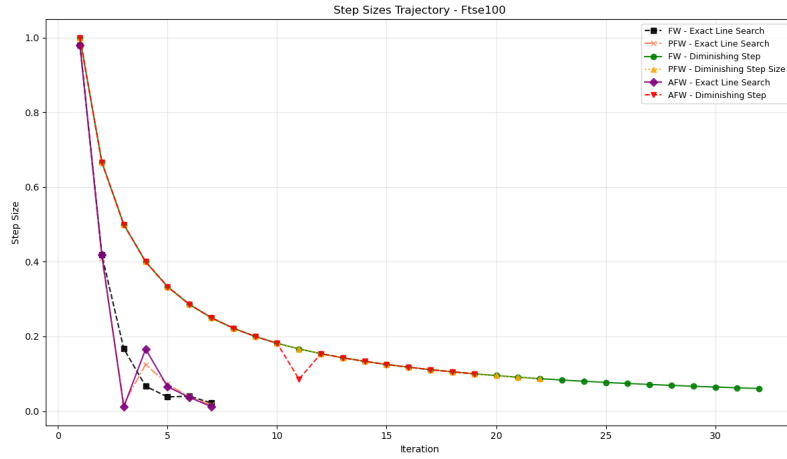


Figure 52: Stepsizes Trajectories for FW, AFW, PFW on FTSE100

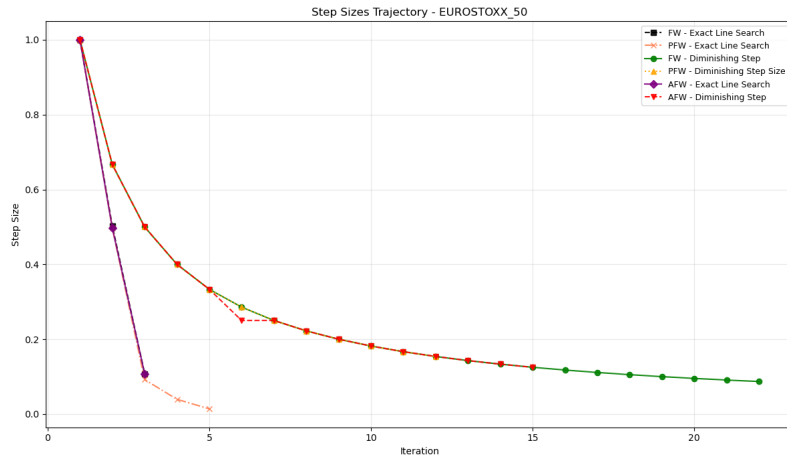


Figure 53: Stepsizes Trajectories for FW,AFW,PFW on EUROSTOXX50

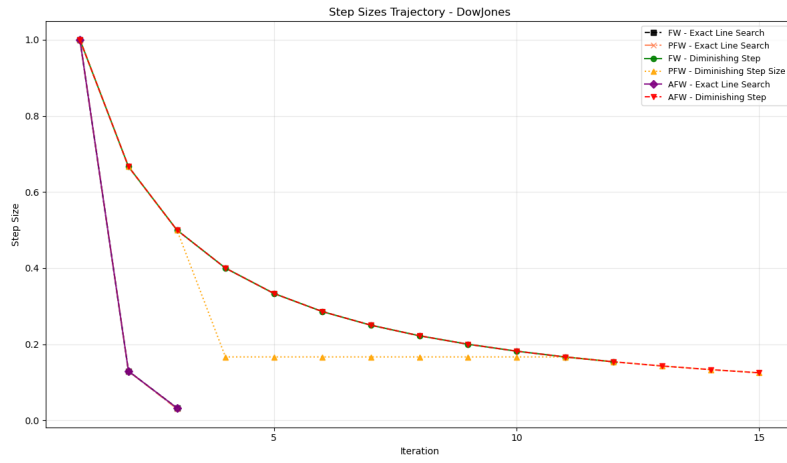


Figure 54: Stepsizes Trajectories for FW,AFW,PFW on DOWJONES

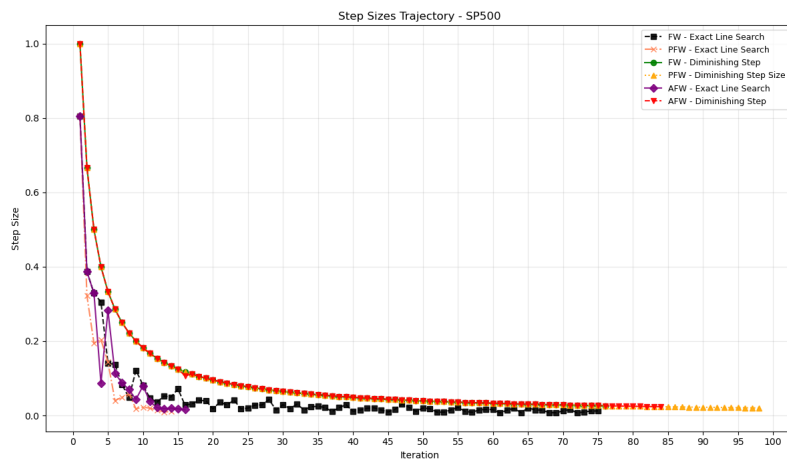


Figure 55: Stepsizes Trajectories for FW,AFW,PFW on S&P500

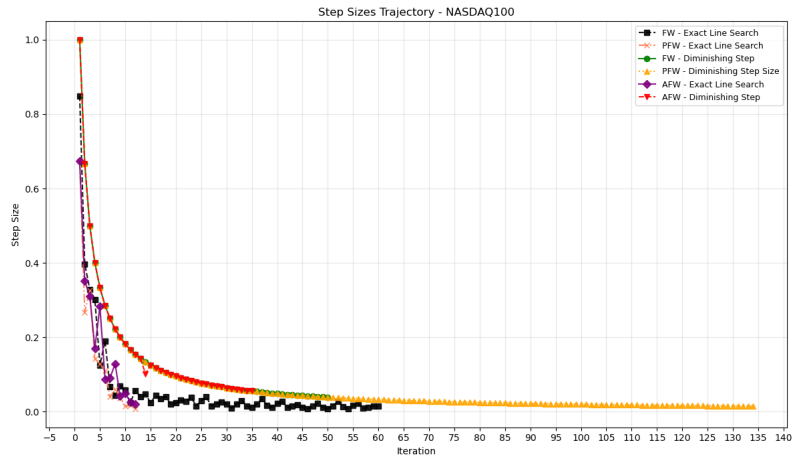


Figure 56: Stepsizes Trajectories for FW,AFW,PFW on NASDAQ100

7.5 CPU-time comparisons

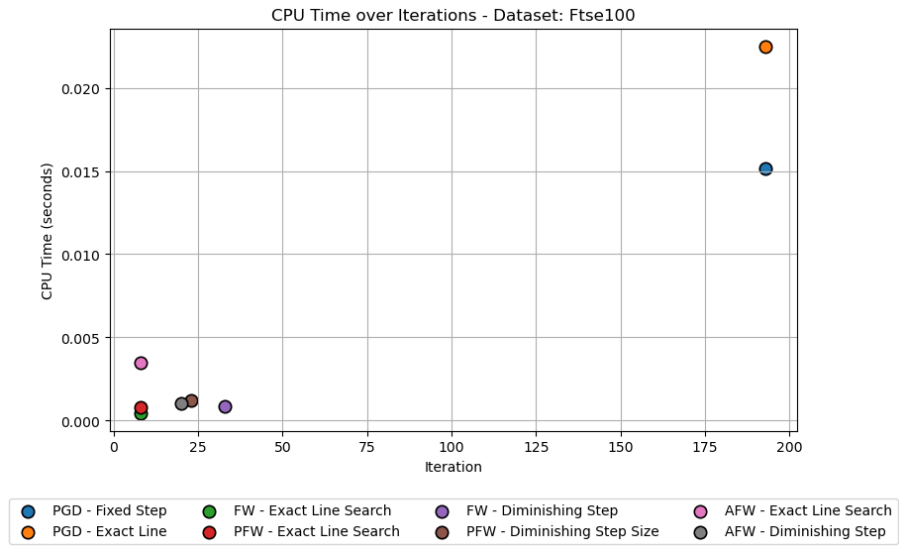


Figure 57: CPU time of all methods on FTSE100

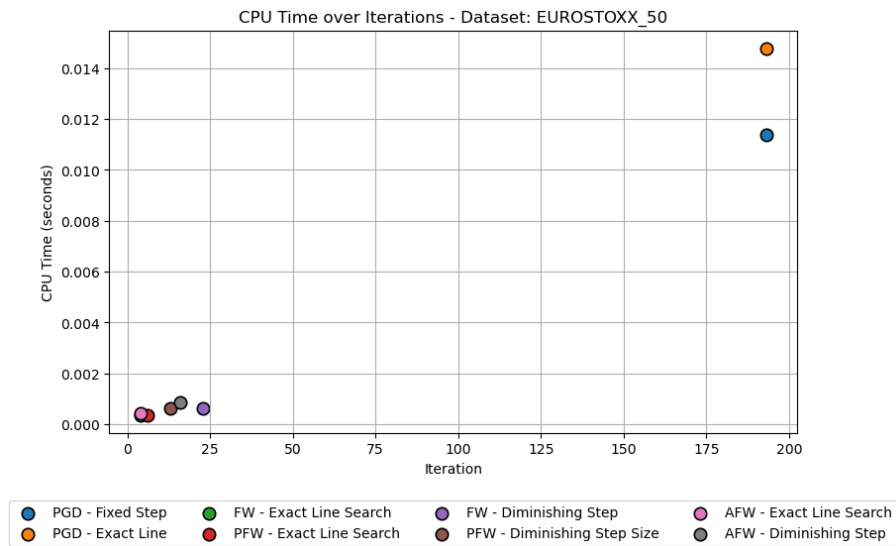


Figure 58: CPU time of all methods on EUROSTOXX50

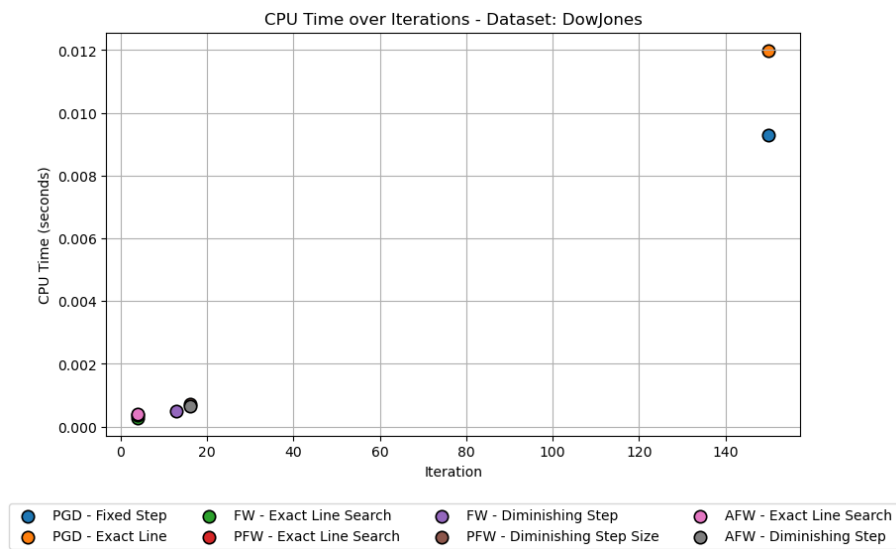


Figure 59: CPU time of all methods on DOWJONES

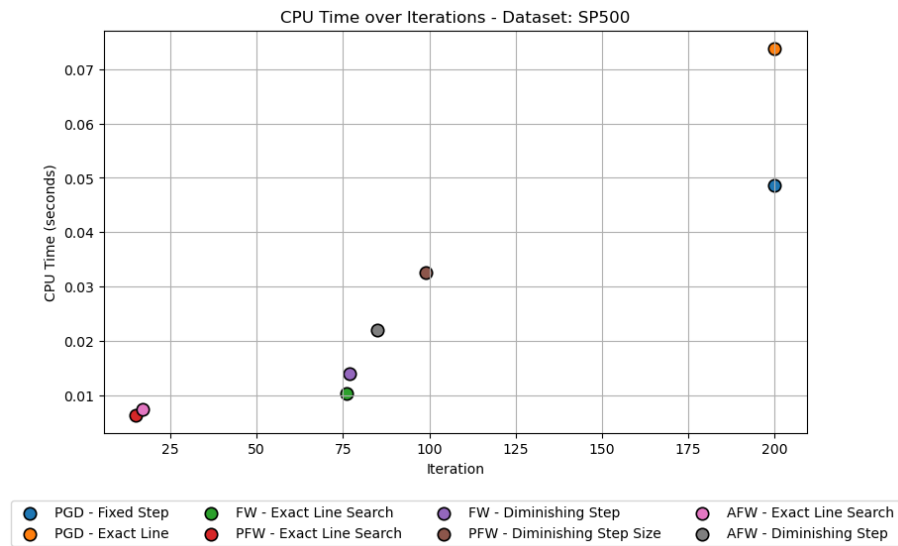


Figure 60: CPU time of all methods on S&P500

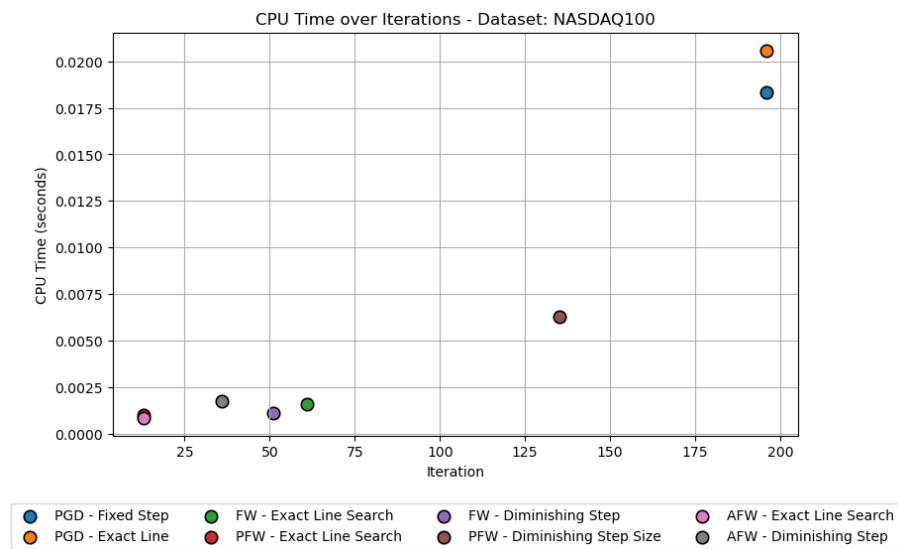


Figure 61: CPU time of all methods on NASDAQ100

7.6 Portfolio Allocations

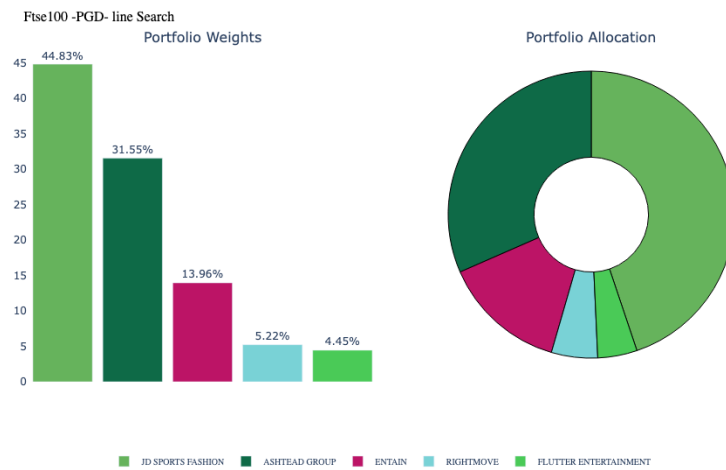


Figure 62: Portfolio allocation on FTSE100 with PGD-line search

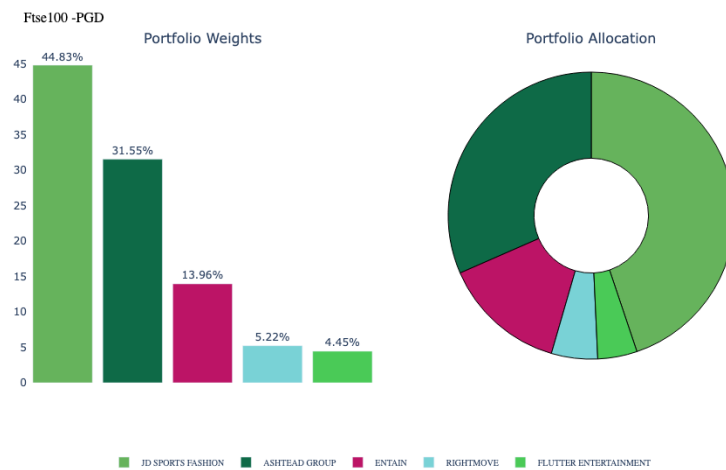


Figure 63: Portfolio allocation on FTSE100 with PGD-fixed step

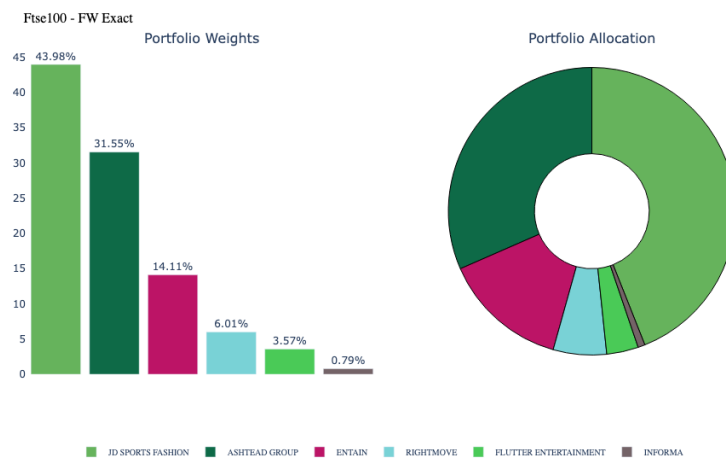


Figure 64: Portfolio allocation on FTSE100 with FW-line search

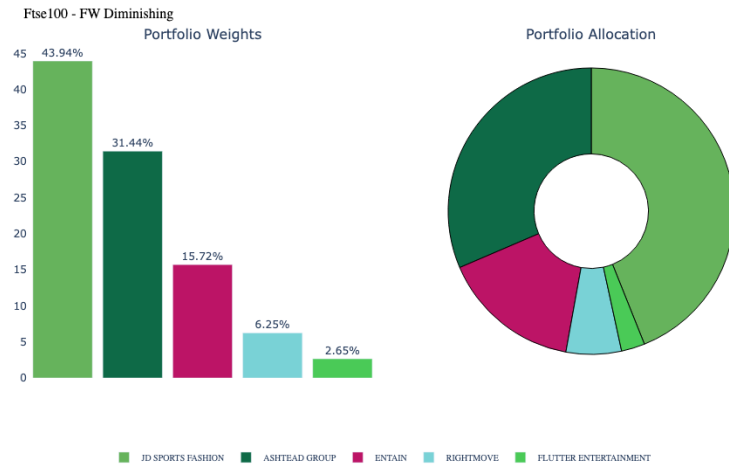


Figure 65: Portfolio allocation on FTSE100 with FW-diminishing rule

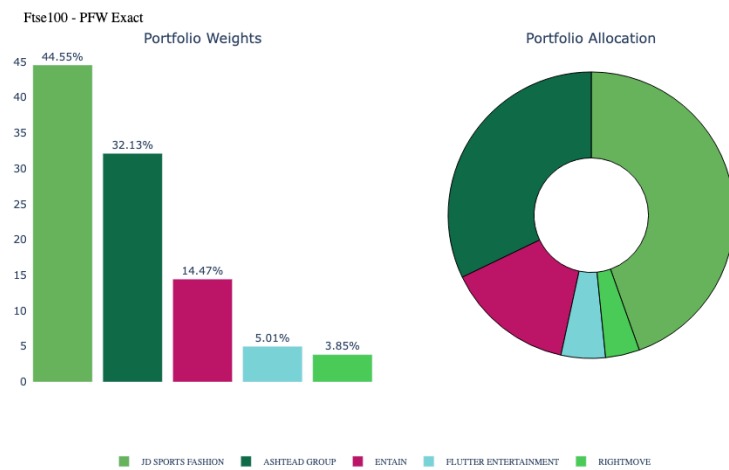


Figure 66: Portfolio allocation on FTSE100 with PFW-line search

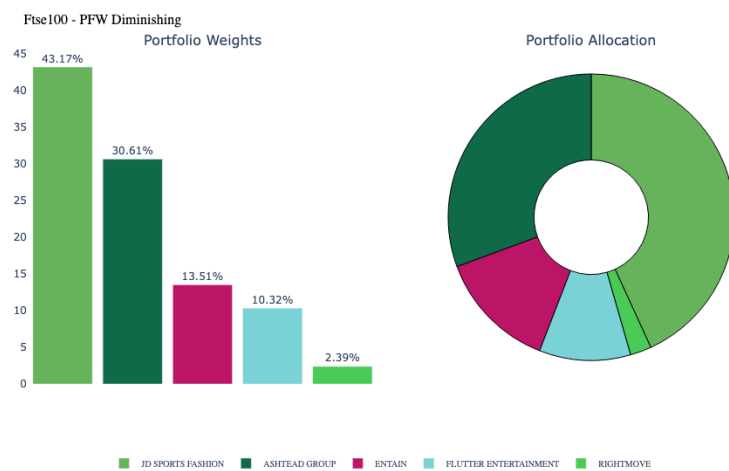


Figure 67: Portfolio allocation on FTSE100 with PFW-diminishing rule

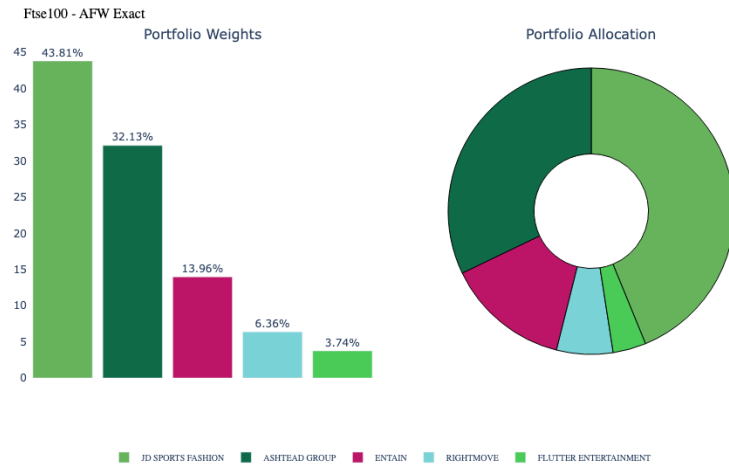


Figure 68: Portfolio allocation on FTSE100 with AFW-line search

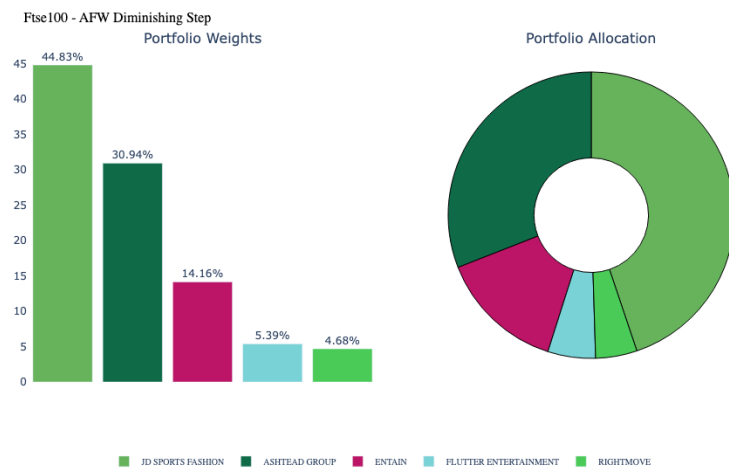


Figure 69: Portfolio allocation on FTSE100 with AFW-diminishing rule

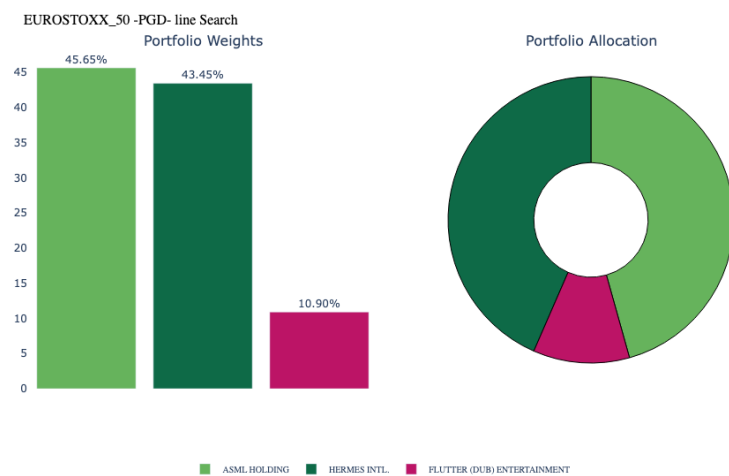


Figure 70: Portfolio allocation on EUROSTOXX50 with PGD-line search

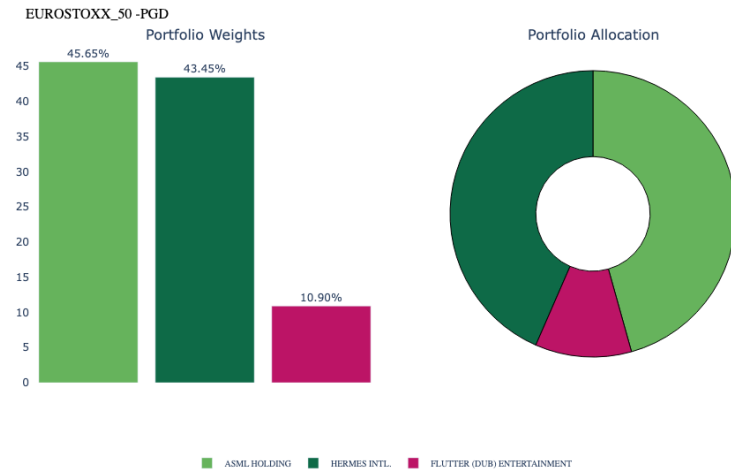


Figure 71: Portfolio allocation on EUROSTOXX50 with PGD-fixed step

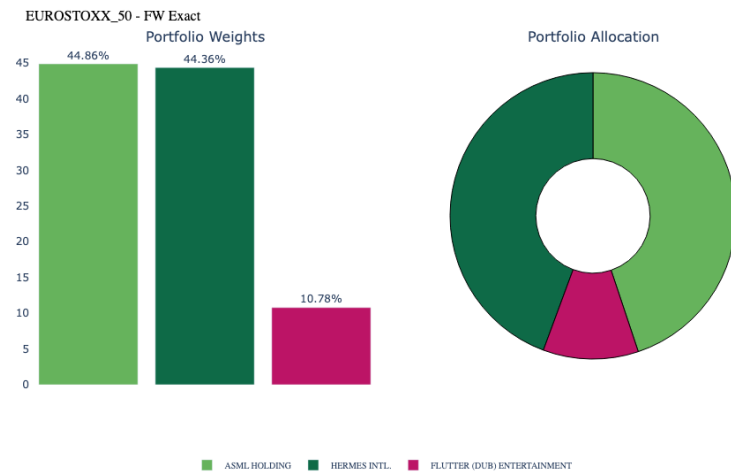


Figure 72: Portfolio allocation on EUROSTOXX50 with FW-line search

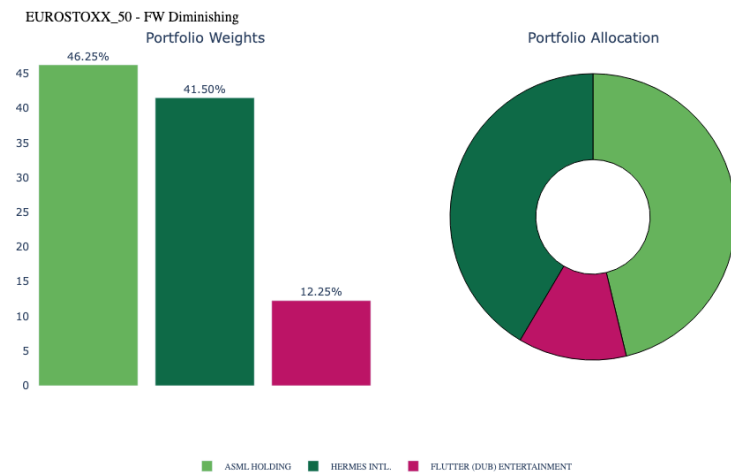


Figure 73: Portfolio allocation on EUROSTOXX50 with FW-diminishing rule

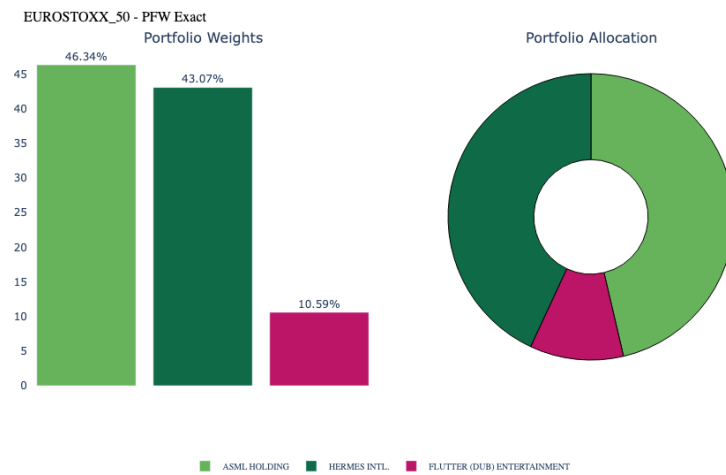


Figure 74: Portfolio allocation on EUROSTOXX50 with PFW-line search

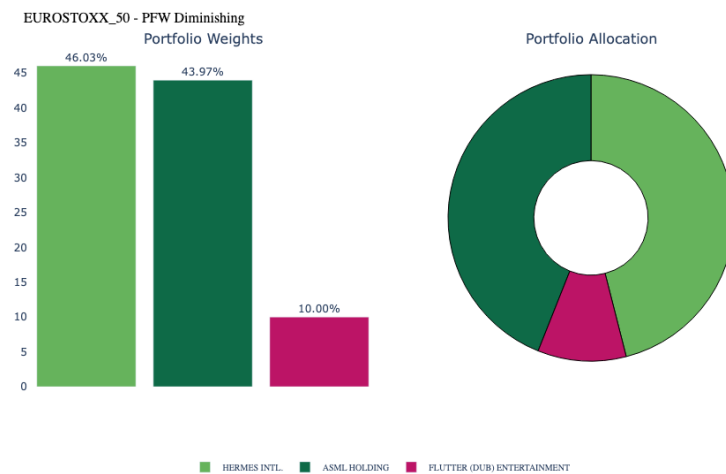


Figure 75: Portfolio allocation on EUROSTOXX50 with PFW-diminishing rule

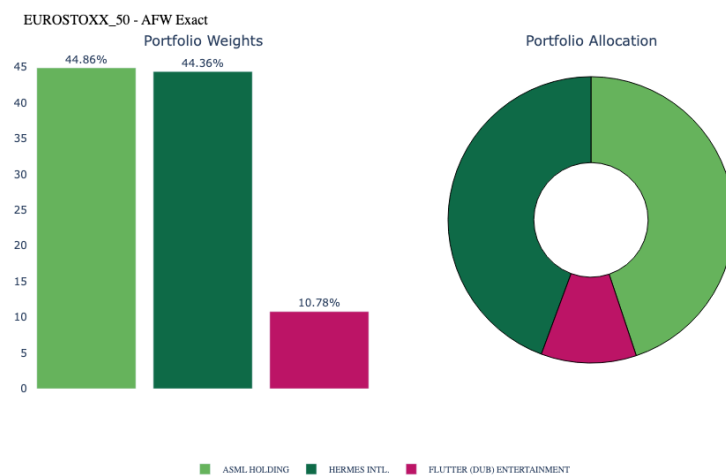


Figure 76: Portfolio allocation on EUROSTOXX50 with AFW-line search

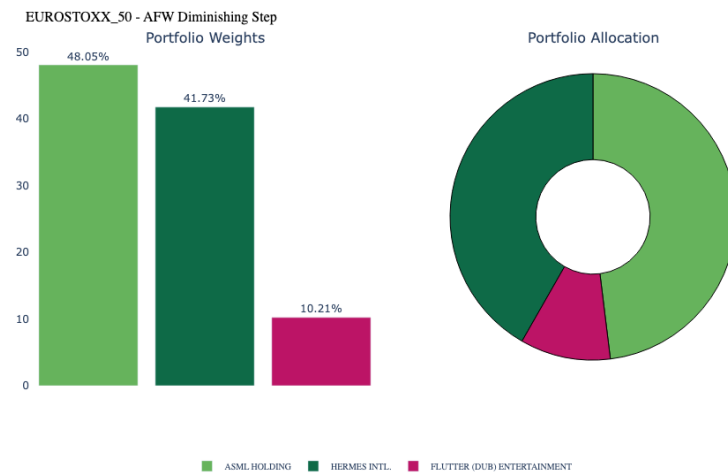


Figure 77: Portfolio allocation on EUROSTOXX50 with AFW-diminishing rule

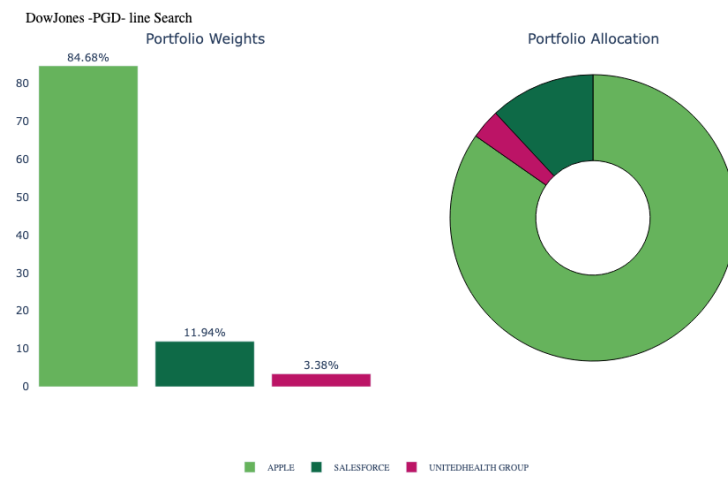


Figure 78: Portfolio allocation on DOWJONES with PGD-line search

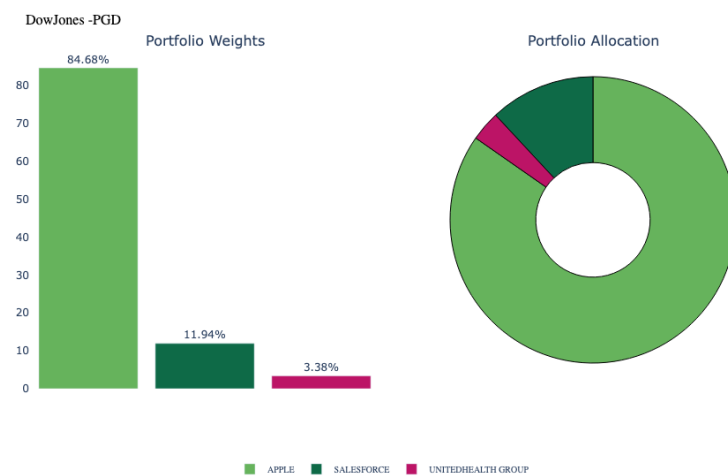


Figure 79: Portfolio allocation on DOWJONES with PGD-fixed step

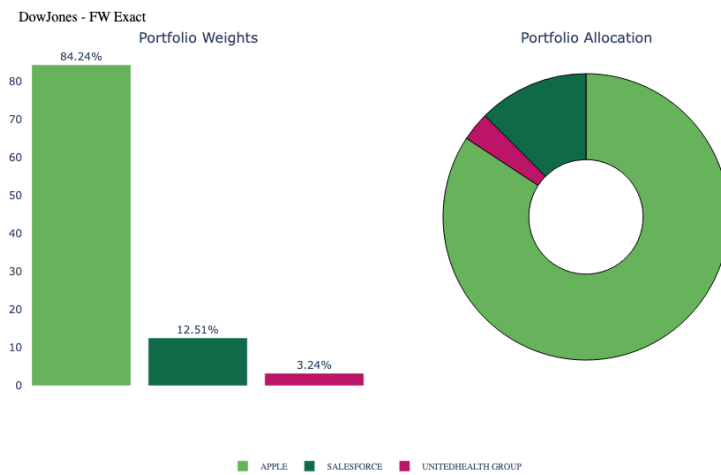


Figure 80: Portfolio allocation on DOWJONES with FW-line search

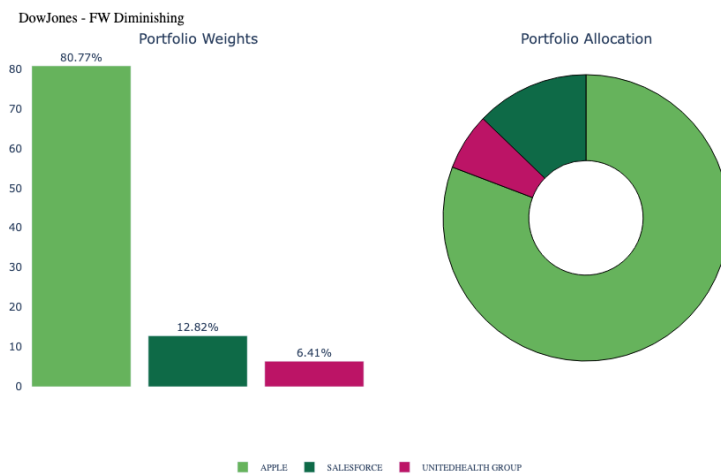


Figure 81: Portfolio allocation on DOWJONES with FW-diminishing rule

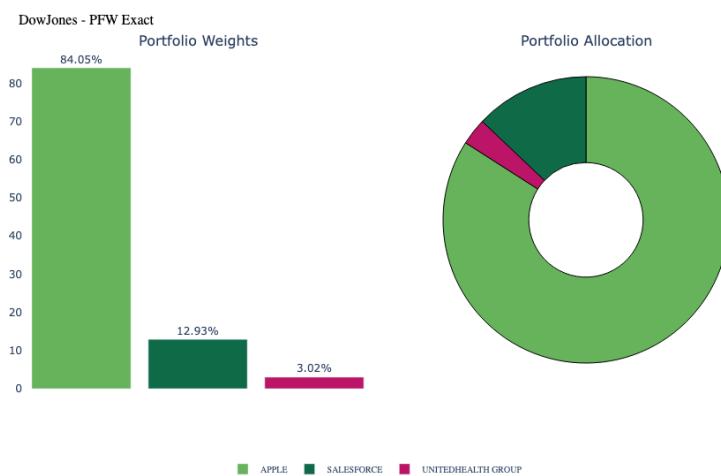


Figure 82: Portfolio allocation on DOWJONES with PFW-line search

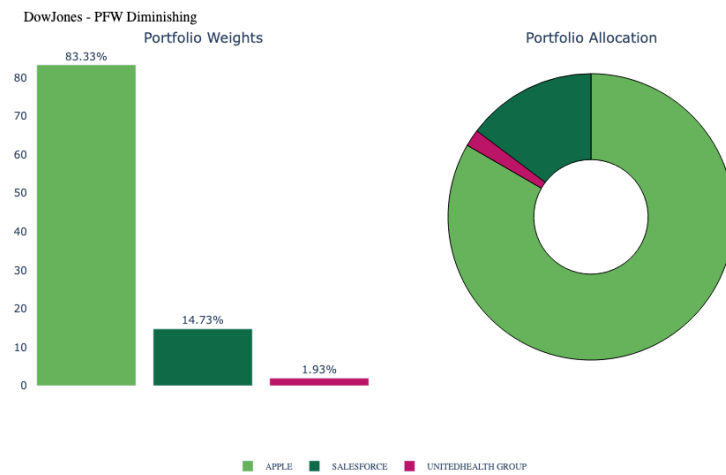


Figure 83: Portfolio allocation on DOWJONES with PFW-diminishing rule

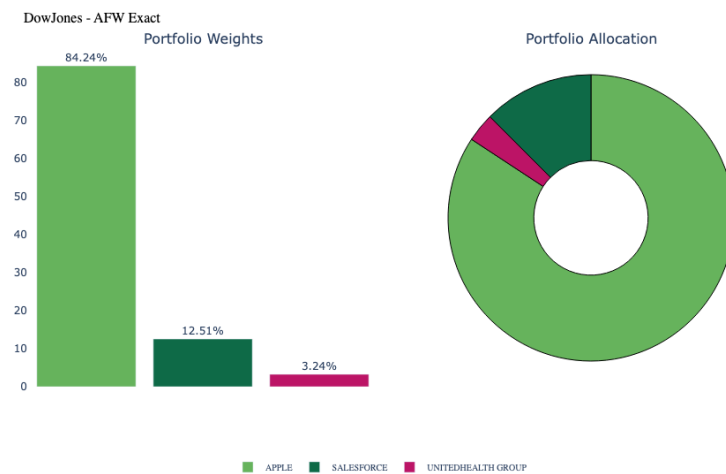


Figure 84: Portfolio allocation on DOWJONES with AFW-line search

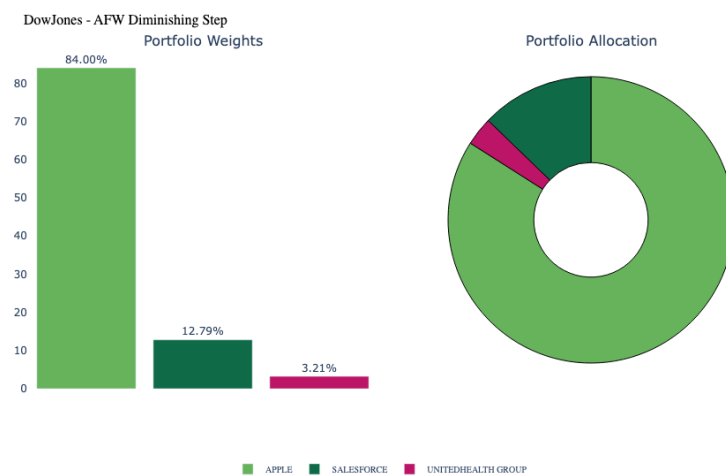


Figure 85: Portfolio allocation on DOWJONES with AFW-diminishing rule

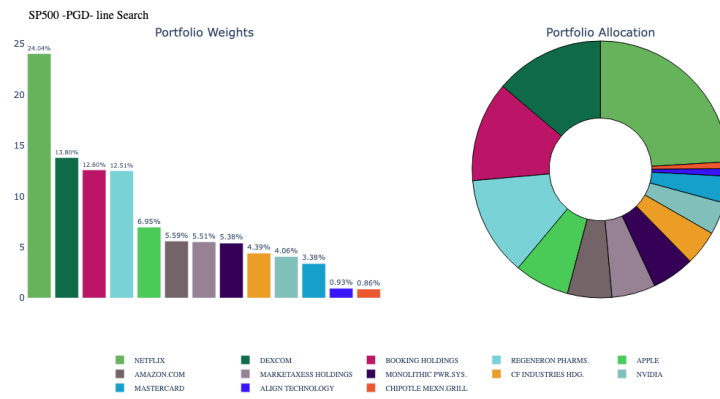


Figure 86: Portfolio allocation on S&P500 with PGD-line search

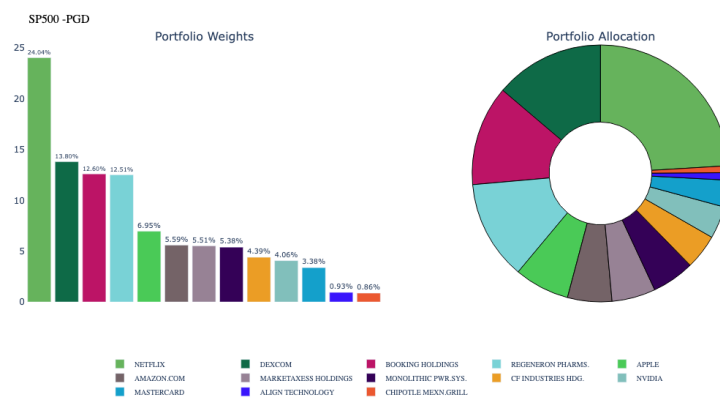


Figure 87: Portfolio allocation on S&P500 with PGD-fixed step

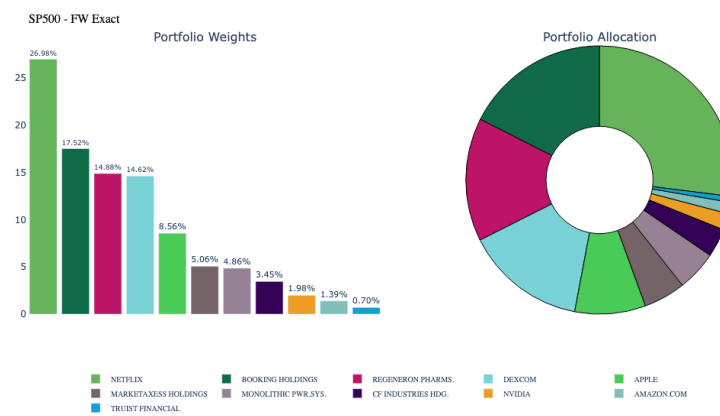


Figure 88: Portfolio allocation on S&P500 with FW-line search

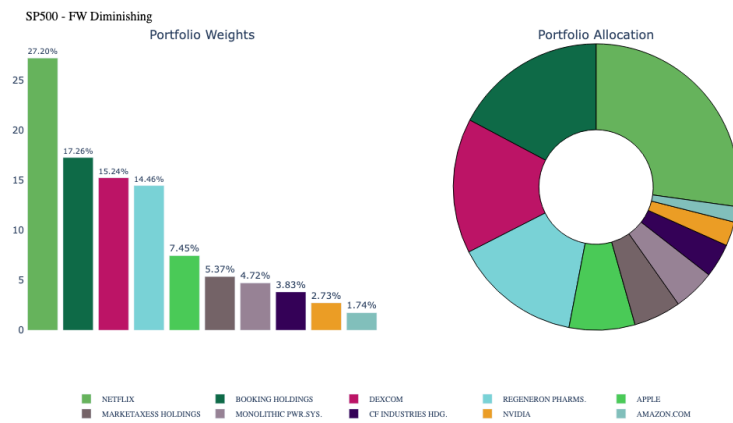


Figure 89: Portfolio allocation on S&P500 with FW-diminishing rule

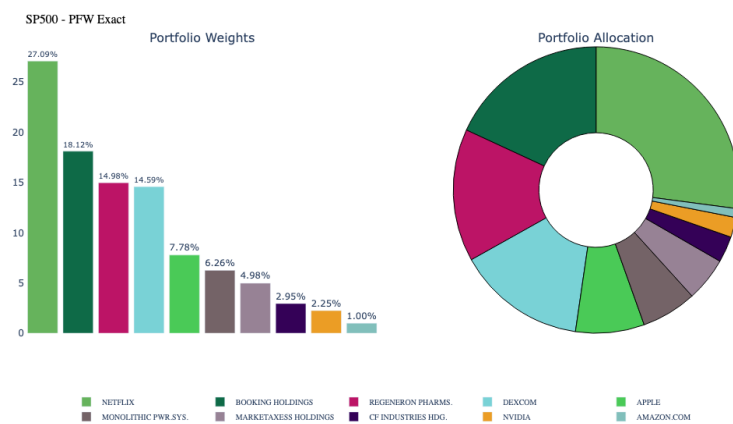


Figure 90: Portfolio allocation on S&P500 with PFW-line search

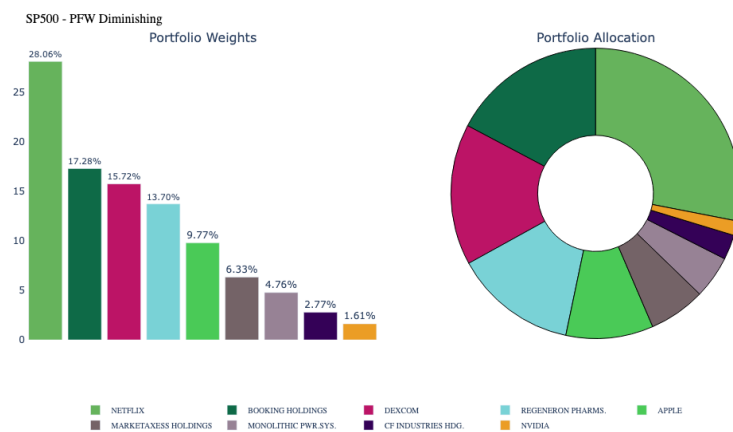


Figure 91: Portfolio allocation on S&P500 with PFW-diminishing rule

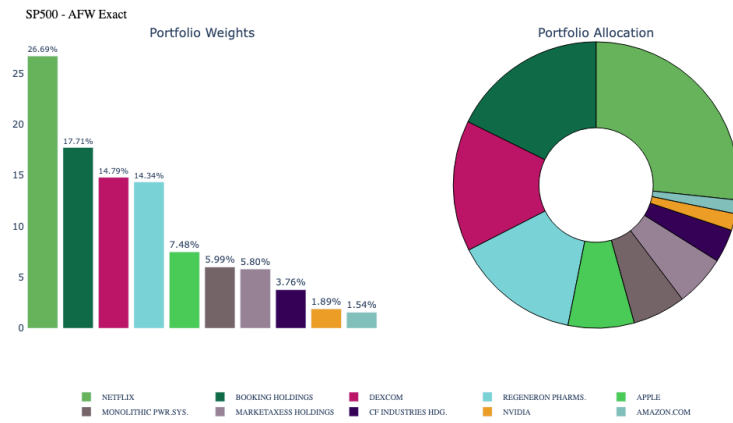


Figure 92: Portfolio allocation on S&P500 with AFW-line search

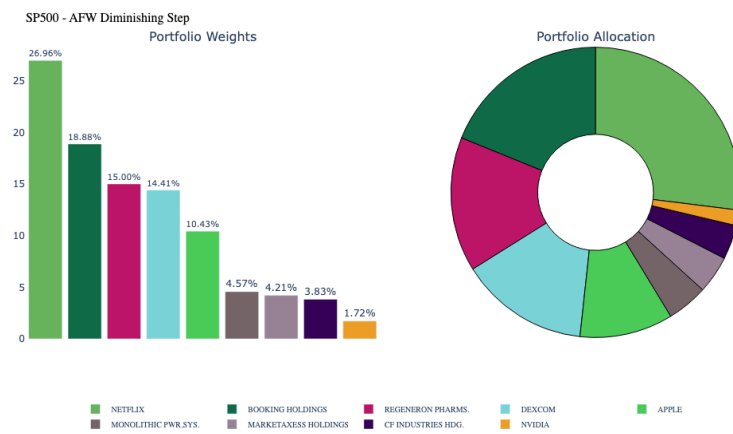


Figure 93: Portfolio allocation on S&P500 with AFW-diminishing rule

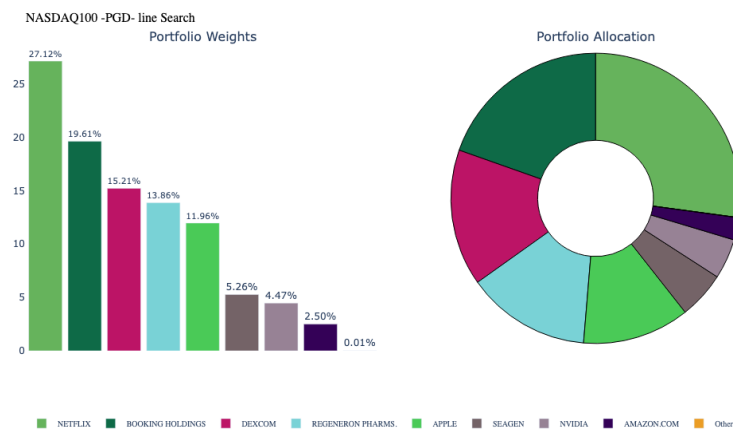


Figure 94: Portfolio allocation on NASDAQ100 with PGD-line search

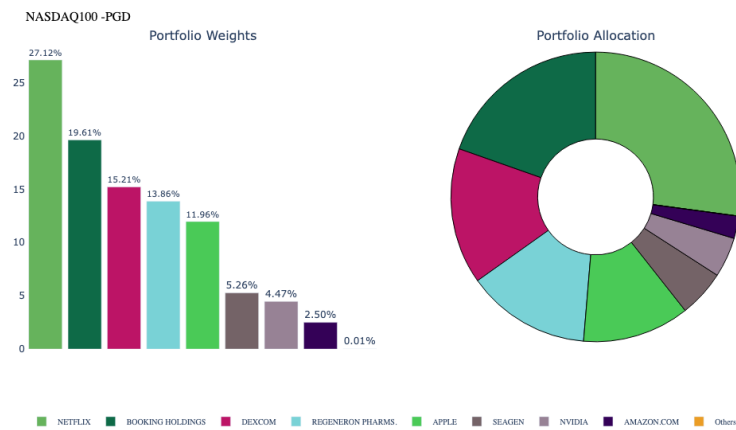


Figure 95: Portfolio allocation on NASDAQ100 with PGD-fixed step

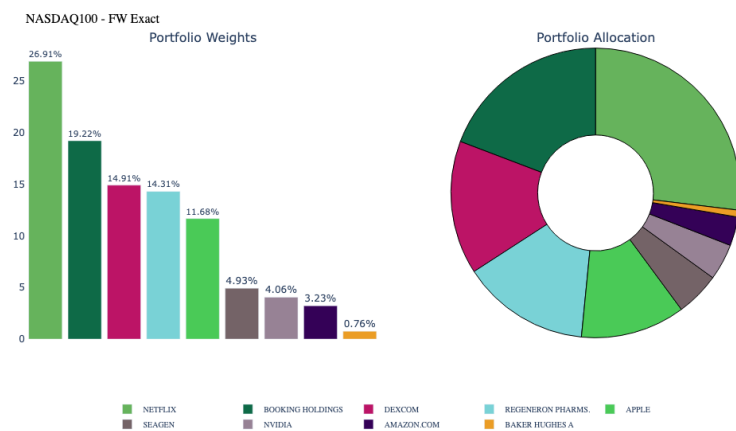


Figure 96: Portfolio allocation on NASDAQ100 with FW-line search

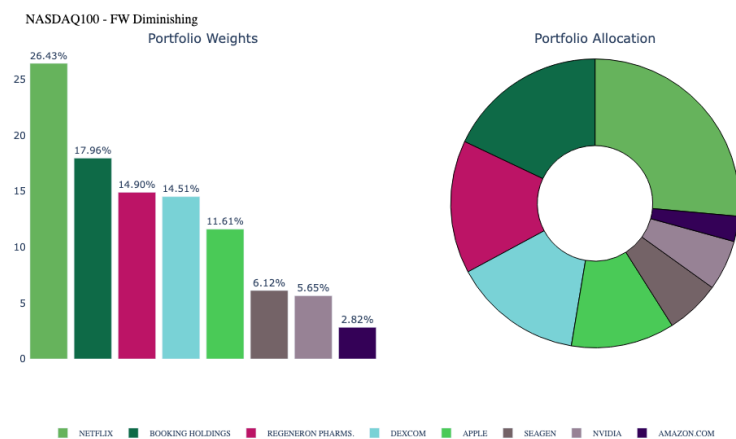


Figure 97: Portfolio allocation on NASDAQ100 with FW-diminishing rule

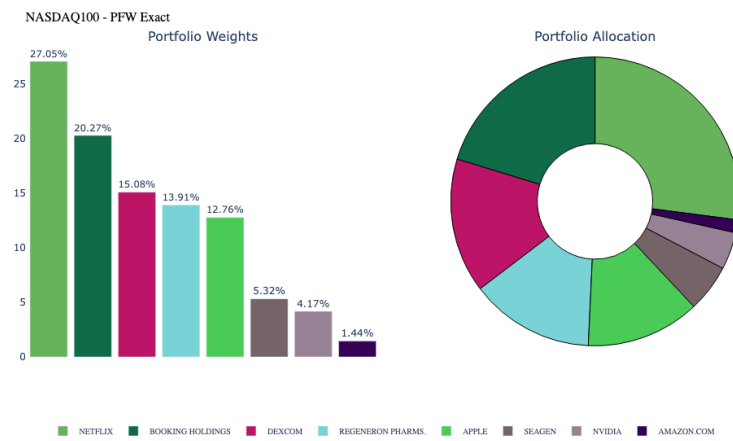


Figure 98: Portfolio allocation on NASDAQ100 with PFW-line search

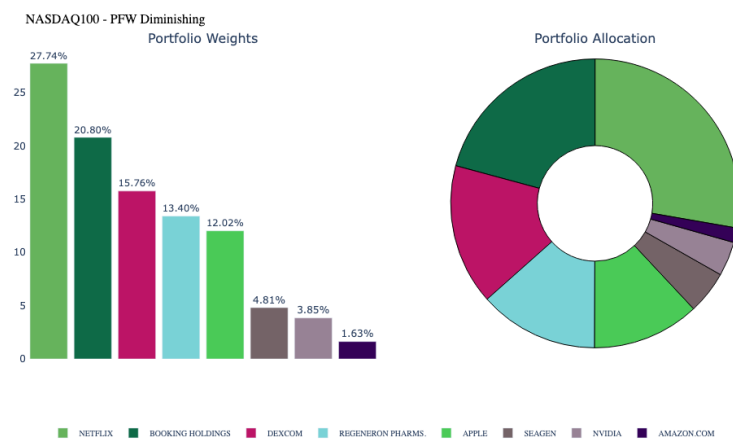


Figure 99: Portfolio allocation on NASDAQ100 with PFW-diminishing rule

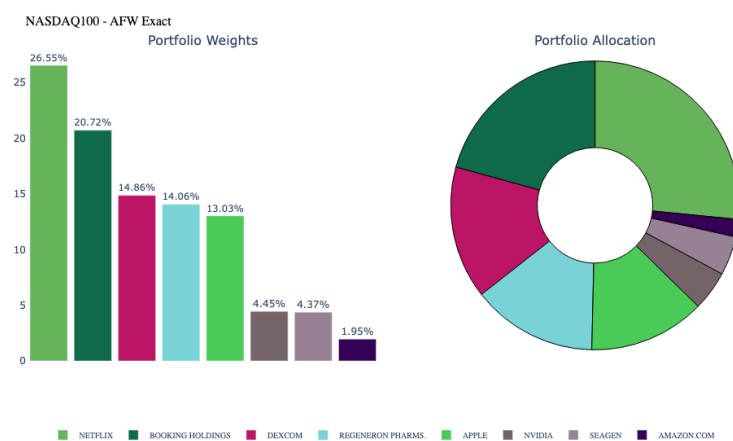


Figure 100: Portfolio allocation on NASDAQ100 with AFW-line search

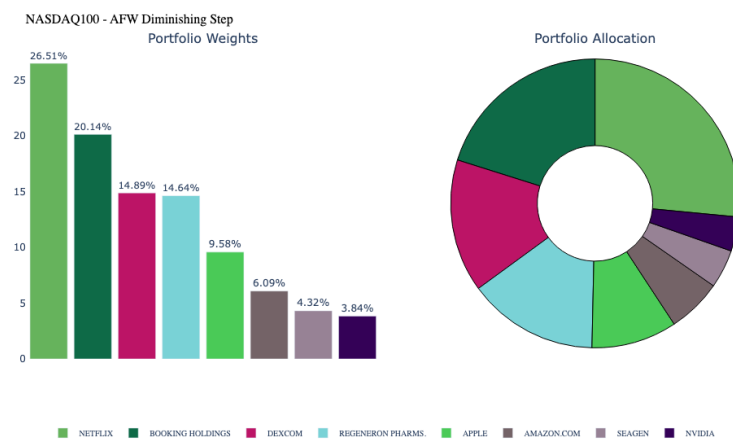


Figure 101: Portfolio allocation on NASDAQ100 with AFW-diminishing rule

References

- [1] Neculai Andrei. A new gradient descent method for unconstrained optimization. *Submitted for Publication to Journal of Mathematics of Computation*,... [http://www.ici.ro/camo/neculai/newstep.pdf@ 02/22/2005](http://www.ici.ro/camo/neculai/newstep.pdf@02/22/2005)"; also: *ICI Technical Report*, 2004.
- [2] Immanuel M Bomze, Francesco Rinaldi, and Damiano Zeffiro. Frank–wolfe and friends: a journey into projection-free first-order optimization methods. *4OR*, 19(3):313–345, 2021.
- [3] Francesco Cesarone. Data sets for portfolio optimization and financial experiments. <https://www.francescocesarone.com/data-sets>, 2025. Accessed: 2025-2/7/.
- [4] Laurent Condat. Fast projection onto the simplex and the l1 ball. *Mathematical Programming*, 158(1):575–585, 2016.
- [5] John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections onto the l1-ball for learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning*, pages 272–279. ACM, 2008.
- [6] Marguerite Frank and Philip Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly*, 3(1-2):95–110, 1956.
- [7] Charles Guille-Escuret, Manuela Girotti, Baptiste Goujaud, and Ioannis Mitliagkas. A study of condition numbers for first-order optimization. In *International Conference on Artificial Intelligence and Statistics*, pages 1261–1269. PMLR, 2021.
- [8] Martin Jaggi. Revisiting frank-wolfe: Projection-free sparse convex optimization. In *International Conference on Machine Learning*, pages 427–435. PMLR, 2013.
- [9] Simon Lacoste-Julien and Martin Jaggi. On the global linear convergence of frank-wolfe optimization variants. *Advances in Neural Information Processing Systems*, 28, 2015.
- [10] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952. doi: 10.2307/2975974. URL <https://doi.org/10.2307/2975974>.
- [11] Francesco Rinaldi and Gennaro Auricchio. Optimization for data science – lecture notes. University of Padova course material, 2024.